

Unidade IV

7 DEVOPS: CONCEITOS E PRÁTICAS

O termo DevOps vem da junção das palavras em inglês *development* e *operations*, que significam "desenvolvimento" e "operações", respectivamente. É uma prática de construção de software que busca, portanto, unir essas duas áreas, automatizando os processos operacionais em todas as fases da engenharia do software. Conforme acentuam Muniz *et al.* (2021), a qualquer processo de software pode ser aplicada a agilidade de forma que a equipe do projeto possa adaptar tarefas e melhorar, guiar o planejamento e reduzir tudo, com exceção do trabalho mais essencial, mantendo a simplicidade e enfatizando a estratégia da entrega incremental.

Com a evolução tecnológica, o software passou a ocupar um papel essencial nas atividades cotidianas de empresas e pessoas. Situações comuns como comprar ingressos para um show, planejar uma viagem, reservar hotel, comprar passagens aéreas, realizar pagamentos, enviar mensagens ou compartilhar fotos envolvem a utilização de diversos sistemas de software. Quando algum desses sistemas apresenta falhas ou fica indisponível, os impactos vão além das organizações que deixam de realizar negócios, pois também afetam os usuários, que ficam impedidos de concluir suas atividades. Diante disso, torna-se fundamental garantir a qualidade, confiabilidade e estabilidade do software em todas as etapas do seu ciclo de vida, desde o desenvolvimento inicial até sua operação em ambiente de produção.

A nuvem tem se consolidado como um importante facilitador para a implementação de metodologias ágeis e práticas de DevOps, permitindo ciclos de desenvolvimento mais curtos, maior automação de processos e respostas mais rápidas às mudanças do mercado. O DevOps não apenas acelera o processo de desenvolvimento, mas também contribui para a melhoria da qualidade do software, da estabilidade dos sistemas em produção e da capacidade de inovação das empresas em um ambiente cada vez mais dinâmico e competitivo.

Adiante, vamos abordar em pormenores a evolução, os fundamentos e as ferramentas de DevOps.

7.1 Evolução de desenvolvimento para o DevOps

Vimos que o software assume um papel estratégico no contexto atual da computação, pois é por meio dele que as necessidades dos usuários são efetivamente atendidas e que o valor da tecnologia se concretiza. Principalmente no contexto de computação em nuvem e DevOps, o software deixa de ser apenas um produto final e passa a ser um elemento em constante evolução, capaz de se adaptar rapidamente às mudanças de negócio. Essa centralidade exige que o desenvolvimento vá além da simples implementação de funcionalidades, incorporando preocupações como escalabilidade, resiliência, segurança e facilidade de manutenção.

Sob essa perspectiva, torna-se evidente que não basta investir apenas em infraestrutura robusta ou em arquiteturas bem planejadas. Embora esses elementos sejam fundamentais para sustentar os sistemas, eles, por si só, não geram valor se não estiverem alinhados a um software que atenda às demandas reais dos usuários. A infraestrutura fornece os recursos, e a arquitetura organiza esses recursos, mas é o software que utiliza essa base para entregar soluções concretas, transformando capacidade técnica em resultados práticos.

O desenvolvimento de software, portanto, passa a ter ainda mais importância e a exigir uma abordagem mais integrada com o ambiente em que será executado. Aplicações modernas precisam ser projetadas para funcionar em ambientes distribuídos, elásticos e altamente dinâmicos, características típicas da nuvem. Isso implica pensar desde o início em aspectos como CI, CD e observabilidade, garantindo que o software possa evoluir de forma rápida e segura ao longo do tempo.

A evolução no desenvolvimento de software pode ser entendida como uma resposta às limitações de abordagens anteriores. A figura 36 ilustra a evolução das abordagens de desenvolvimento de software ao longo do tempo. Inicialmente, o modelo em cascata representa um processo linear e sequencial, em que cada etapa precisa ser concluída antes da próxima começar. Em seguida, o modelo ágil surge como uma alternativa mais flexível, com desenvolvimento iterativo, foco em entregas contínuas e adaptação a mudanças. Por fim, o DevOps amplia essa evolução ao integrar desenvolvimento e operações, promovendo automação, colaboração contínua e entregas mais rápidas e frequentes de software.



Figura 36 – Evolução do desenvolvimento para DevOps

Adaptada de: Santos (2022).

O modelo em cascata foi um dos primeiros processos formais de desenvolvimento de software. Ele é baseado em uma abordagem linear e sequencial, na qual o projeto passa por fases bem definidas: levantamento de requisitos, design, implementação, testes, implantação e manutenção. Cada fase deve ser concluída antes de iniciar a próxima, o que significa que há pouca ou nenhuma sobreposição entre elas. Essa metodologia é bastante estruturada e fácil de entender, sendo adequada para projetos com requisitos bem definidos e pouco propensos a mudanças. No entanto, sua rigidez é uma grande limitação, pois qualquer alteração nos requisitos após o início do projeto pode ser difícil e custosa de implementar. Além disso, o modelo não favorece a adaptação às mudanças durante o processo, o que pode resultar em produtos que não atendem completamente às necessidades do cliente ou do mercado, especialmente em ambientes dinâmicos.

As principais vantagens do modelo em cascata são: processo bem estruturado e fácil de gerenciar; documentação detalhada de cada fase; e boa abordagem para projetos com requisitos bem definidos e estáveis.

Vemos que o modelo em cascata pode ser indicado para determinados tipos de projetos, especialmente aqueles em que os requisitos são bem definidos, estáveis e pouco propensos a mudanças ao longo do desenvolvimento. Por exemplo, projetos de engenharia, construção civil ou desenvolvimento de software para setores altamente regulamentados, como o setor aeroespacial ou militar, muitas vezes se beneficiam do uso do modelo em cascata. Nessas situações, a documentação detalhada, o planejamento rigoroso e a execução sequencial ajudam a garantir que todas as etapas sejam concluídas de forma controlada, atendendo a requisitos específicos e garantindo conformidade com normas e padrões rígidos. Além disso, projetos com escopo bem definido, em que o cliente tem uma compreensão clara do produto final desde o início, também podem se beneficiar do modelo em cascata, pois ele permite um controle mais rígido do cronograma e do orçamento, facilitando a gestão de riscos em ambientes altamente regulamentados ou com alta complexidade técnica.

Todavia, as desvantagens do modelo em cascata são: rígido e inflexível a mudanças; dificuldade de adaptação a requisitos que evoluem; e problemas só aparecem na fase de testes ou implantação, dificultando a correção de erros precocemente.

A figura a seguir ilustra o modelo de desenvolvimento em cascata.

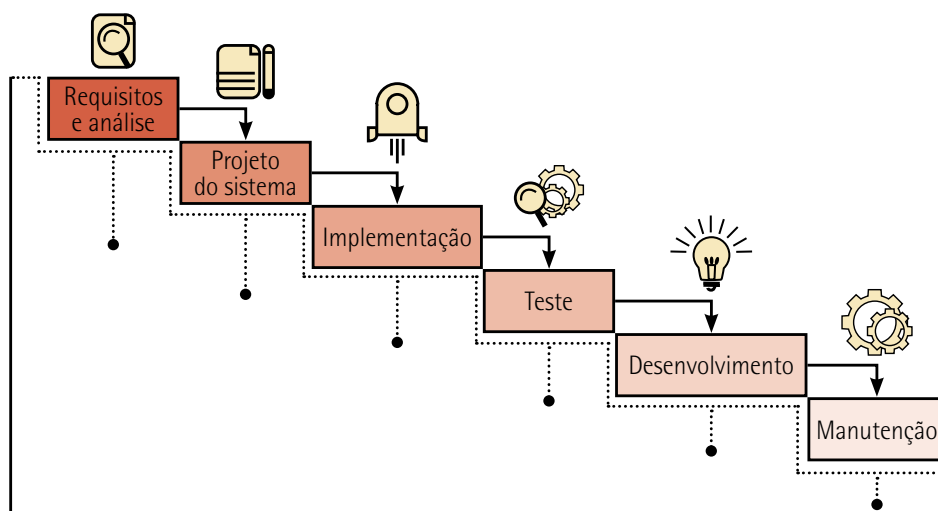


Figura 37 – Modelo de desenvolvimento em cascata

Fonte: Mignon (2025, p. 2).

Em 2001, um grupo de desenvolvedores elaborou o Manifesto Ágil, estabelecendo 12 princípios voltados à gestão de projetos de software. A proposta surgiu como uma alternativa aos métodos tradicionais, considerados rígidos e burocráticos, trazendo uma abordagem mais leve e adaptável. O foco passou a ser a entrega rápida e contínua de valor ao cliente. Além disso, destacou-se a relevância de equipes enxutas, colaborativas e motivadas. Esses princípios servem de base para metodologias como scrum e extreme programming (XP), que priorizam flexibilidade, adaptação constante e atendimento às reais necessidades dos usuários. Cabe ressaltar que marcos importantes na evolução do DevOps tiveram origem dentro da própria comunidade ágil (Mignon, 2025).

O agile, ou desenvolvimento ágil, surgiu como uma resposta às limitações do modelo em cascata, promovendo uma abordagem mais flexível, iterativa e colaborativa. Em vez de seguir um plano rígido, as equipes ágeis trabalham em ciclos curtos chamados sprints, nos quais entregam funcionalidades completas e utilizáveis.

Essa metodologia valoriza a comunicação contínua com o cliente, permitindo que mudanças nos requisitos sejam incorporadas ao longo do projeto, sem grandes prejuízos. O foco está na entrega frequente de valor, na adaptação rápida às mudanças e na colaboração entre equipes multidisciplinares. Assim, o produto final tende a ser mais alinhado às necessidades reais do cliente, além de permitir ajustes constantes ao longo do desenvolvimento. É uma abordagem especialmente útil em ambientes nos quais os requisitos podem evoluir rapidamente ou em que a inovação é uma prioridade.

As principais vantagens do agile são: flexibilidade para mudanças durante o projeto; entregas frequentes de funcionalidades, facilitando feedback e ajustes; maior envolvimento do cliente e das equipes de desenvolvimento; e melhor adaptação às mudanças de mercado ou requisitos.

O agile pode ser a abordagem mais recomendada em diversos casos, especialmente aqueles em que a flexibilidade, a adaptação às mudanças e a CD de valor são essenciais. Projetos em ambientes dinâmicos, como desenvolvimento de software para startups, produtos inovadores ou projetos em setores de tecnologia em que os requisitos podem evoluir rapidamente, se beneficiam bastante do agile. Além disso, quando o cliente ou o mercado exige entregas frequentes, feedback constante e ajustes rápidos, esse método se mostra uma escolha eficiente. Ele também é indicado em projetos nos quais a colaboração entre equipes multidisciplinares é fundamental, permitindo que diferentes áreas trabalhem juntas de forma mais integrada e responsiva. Portanto, sempre que houver necessidade de adaptação rápida, inovação contínua ou envolvimento ativo do cliente durante o processo, o agile tende a ser a melhor opção.

Em contrapartida, o agile apresenta algumas desvantagens. São elas: pode ser difícil de gerenciar em equipes grandes ou com pouca experiência; requer maior disciplina e comunicação constante; e pode gerar escopo variável se não houver um bom controle.



Saiba mais

Para saber mais sobre o Manifesto Ágil, seus 12 princípios e o contexto histórico de sua criação, acesse o site a seguir:

MANIFESTO para desenvolvimento ágil de software. *Agile Manifesto*, [s.d.]. Disponível em: <https://tinyurl.com/3z3xj7dy>. Acesso em: 1 jun. 2026.

Entre os anos de 2007 e 2008, surgiu o movimento DevOps com o objetivo de reduzir o conflito tradicional entre as equipes de desenvolvimento, que buscam mudanças rápidas e frequentes, e as equipes de operações, que priorizam estabilidade e confiabilidade. Para isso, esta nova abordagem promove colaboração, automação e CI, permitindo entregar mudanças com mais segurança e estabilidade.

O DevOps representa uma evolução na cultura de desenvolvimento de software, buscando integrar de forma mais eficiente as equipes de desenvolvimento (Dev) e operações (Ops). Essa abordagem visa quebrar as barreiras tradicionais entre essas áreas, promovendo uma colaboração mais estreita, automação de processos e uma cultura de melhoria contínua. O foco do DevOps é acelerar a entrega de software de alta qualidade, utilizando práticas como CI, CD e automação de testes e implantação. Com isso, as equipes podem lançar novas funcionalidades ou correções de forma mais rápida, confiável e frequente, além de monitorar continuamente o desempenho do sistema em produção para responder rapidamente a problemas. O DevOps também incentiva uma mentalidade de aprendizado constante, em que a automação e a cultura de feedback contínuo são essenciais para melhorar processos e produtos.

As principais vantagens do DevOps são: entregas mais rápidas e frequentes; maior confiabilidade e estabilidade do sistema; automação de processos para reduzir erros humanos; melhor colaboração entre equipes de desenvolvimento e operações; e feedback contínuo para melhorias constantes.

Importante pontuar que, embora o DevOps seja considerado uma abordagem altamente eficiente e evoluída para acelerar a entrega de software com maior qualidade e ganhos importantes para as organizações, a sua adoção também impõe desafios significativos. Um deles é a mudança cultural, já que exige maior colaboração entre equipes tradicionalmente separadas nas empresas, como desenvolvimento e operações. Para isso, é necessário romper silos e promover responsabilidade compartilhada. Além disso, há a complexidade na automação de pipelines de CI e CD, que demanda ferramentas adequadas, padronização de processos e maturidade técnica. A gestão da IaC, o monitoramento constante e a necessidade de garantir segurança ao longo de todo o ciclo também aumentam o nível de exigência. Soma-se a isso a dificuldade de capacitação das equipes e a resistência à mudança, tornando a implementação de DevOps um processo gradual, que depende tanto de tecnologia quanto de transformação organizacional.

Portanto, as principais desvantagens do DevOps são: requerer mudança cultural significativa na organização; pode demandar investimentos em automação e treinamento; e implementar práticas de DevOps pode ser complexo inicialmente.

Os sistemas atuais precisam ser desenvolvidos já considerando que irão executar em ambientes descentralizados, com capacidade de expansão automática e sujeitos a mudanças frequentes, características marcantes da computação em nuvem. Por isso, desde o início do processo, é importante adotar práticas que favoreçam a automação, a entrega frequente de novas versões e o acompanhamento contínuo do funcionamento da aplicação, permitindo que o software seja aprimorado de forma ágil, confiável e segura ao longo do tempo.

É justamente nesse contexto que a cultura DevOps se destaca, promovendo a aproximação entre desenvolvimento e operações. Ao integrar essas áreas, busca-se reduzir o tempo entre a criação e a entrega de valor, aumentando a qualidade e a confiabilidade das aplicações. Para profissionais de desenvolvimento de sistemas, compreender essa visão é essencial, pois amplia o papel do desenvolvedor, que deixa de focar apenas no código e passa a atuar de forma mais abrangente em todo o ciclo de vida do software.

O DevOps é representado com uma imagem em formato de infinito, como ilustrado na figura a seguir.

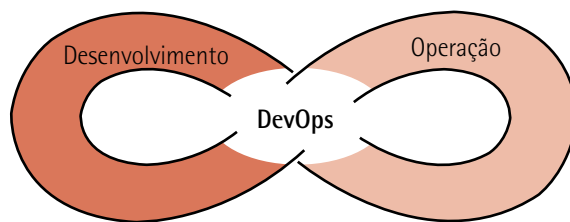


Figura 38 – Símbolo do DevOps

Adaptada de: Mignon (2025).

Esse símbolo do DevOps expressa a natureza contínua, cíclica e integrada do desenvolvimento e das operações de software. Trata-se da busca pela harmonização das duas equipes e um conglomerado de ferramentas e tecnologias que permitem a criação de uma cultura de colaboração para fugir dos desperdícios e impedimentos, automatizando processos repetitivos. O símbolo ressalta que o DevOps não é um processo com um fim definido, mas sim um ciclo constante de melhoria, colaboração e entrega.



Lembrete

São três os modelos de desenvolvimento de software:

- **Cascata:** modelo sequencial e rígido, em que cada fase (requisitos, desenvolvimento e testes) acontece uma única vez, com pouca flexibilidade para mudanças.
- **Ágil:** abordagem iterativa e incremental, focada em entregas frequentes, adaptação rápida e colaboração com o cliente.
- **DevOps:** evolução do ágil que integra desenvolvimento e operações, com automação, CI e CD, visando rapidez, qualidade e estabilidade no ciclo de software.

7.2 Fundamentos de DevOps: princípios, cultura e processos

DevOps consiste em uma abordagem que reúne aspectos culturais, práticas e ferramentas com o propósito de aproximar as equipes de desenvolvimento e operações. Seu foco está em tornar o processo de entrega de software mais ágil, confiável e contínuo, garantindo maior eficiência e qualidade ao longo de todo o ciclo de vida das aplicações.

Muniz *et al.* (2021) explicam que existem três maneiras do DevOps que representam os três princípios fundamentais para implementar e entender o DevOps.

A figura a seguir ilustra as três diferentes maneiras, ou os três princípios, do DevOps:

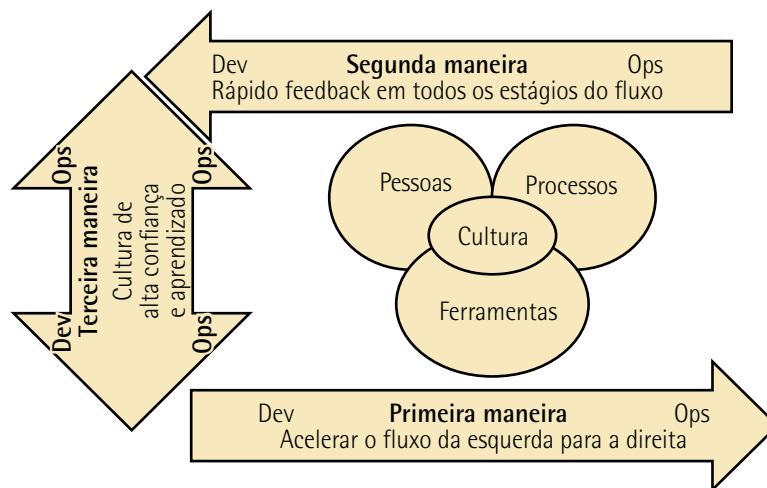


Figura 39 – Três maneiras do DevOps

Adaptada de: Muniz *et al.* (2021).

A primeira maneira do DevOps tem como foco acelerar o fluxo de trabalho desde o desenvolvimento até as operações e, por fim, até o cliente. A ideia central é garantir que o valor gerado percorra todo o sistema de forma rápida, contínua e eficiente, evitando gargalos e retrabalhos. Para isso, considera-se o processo como um todo, desde a concepção da solução até sua entrega, e não apenas etapas isoladas ou departamentos separados.

Esse princípio enfatiza a CD de valor ao cliente por meio da redução de desperdícios, da melhoria do fluxo e da busca constante por qualidade desde a origem. Práticas como trabalhar com pequenos lotes, tornar o trabalho visível, automatizar testes e utilizar IaC contribuem para tornar o processo mais ágil e confiável. Além disso, ajuda a identificar e tratar gargalos, principalmente com adoção de arquiteturas e releases de baixo risco, o que reduz falhas e facilita a implantação contínua.

A segunda maneira do DevOps está centrada na criação de ciclos rápidos de feedback ao longo de todo o fluxo de desenvolvimento e operação. A ideia principal é garantir que informações sobre o funcionamento do sistema, sejam acertos, erros ou oportunidades de melhoria, retornem o mais rápido possível para as equipes, permitindo ajustes contínuos e decisões mais assertivas. Quanto mais rápido esse feedback ocorre, mais ágil é a correção de problemas e a evolução do produto.

Esse princípio promove atuação integrada entre desenvolvimento e operações, incentivando o trabalho colaborativo, com menos barreiras e aprovações excessivas. Práticas como desenvolvimento orientado por hipóteses, revisão de código e atuação conjunta no dia a dia reforçam essa integração. A telemetria desempenha papel fundamental nesse contexto, fornecendo dados por meio de métricas, logs e eventos que permitem monitoramento proativo e análise de riscos. Ao adotar o conceito de telemetria self-service, essas informações ficam acessíveis a todos os envolvidos, democratizando o conhecimento e acelerando o aprendizado.

Outro aspecto relevante é o compartilhamento de responsabilidades, como no plantão 24x7, que envolve diferentes membros do fluxo na resolução de incidentes, promovendo empatia e aprendizado coletivo. Por fim, práticas como as revisões de prontidão contribuem para garantir que o software esteja preparado para produção e manutenção, fortalecendo a confiabilidade do sistema.

A terceira maneira do DevOps tem como foco a criação de uma cultura de aprendizado contínuo, baseada em confiança, colaboração e segurança psicológica. Nesse contexto, entende-se que erros são inevitáveis e devem ser encarados como oportunidades de aprendizado, e não como falhas a serem punidas. Essa abordagem fortalece a cultura DevOps ao incentivar a experimentação, a inovação e a melhoria constante dos processos e produtos.

Para que isso aconteça, é fundamental estabelecer um ambiente seguro, onde as equipes possam assumir riscos de forma controlada. Práticas como reuniões *post-mortem* sem culpabilização permitem analisar incidentes de forma construtiva, buscando entender causas e evitar recorrências. Além disso, a realização de "dias de jogos" (simulações de falhas) e até mesmo a injeção controlada de falhas em produção contribuem para aumentar a resiliência dos sistemas e preparar melhor as equipes para situações reais.

A terceira maneira também enfatiza a disseminação do conhecimento e a transformação de aprendizados locais em melhorias globais, beneficiando toda a organização. Iniciativas como blitz de melhoria contínua e o uso de testes automatizados como forma de documentação reforçam esse processo. Dessa forma, o objetivo é não apenas melhorar sistemas, mas também evoluir continuamente a forma de trabalhar, mantendo viva a cultura DevOps ao longo do tempo.



Lembrete

As três maneiras são, na verdade, três princípios fundamentais do DevOps (ou três formas de pensar/organizar o trabalho em DevOps).

- **Primeira maneira:** fluxo (do desenvolvimento para operações).
- **Segunda maneira:** feedback (de operações para desenvolvimento).
- **Terceira maneira:** aprendizado e melhoria contínua.

A abordagem DevOps tem como principal objetivo aproximar as equipes de desenvolvimento e operações, promovendo uma atuação integrada que permita acelerar a entrega de software e, ao mesmo tempo, elevar seus níveis de qualidade e confiabilidade. Para isso, apoia-se em quatro pilares fundamentais que orientam a forma como o trabalho é organizado e executado.

Apresentamos a seguir os quatro pilares do DevOps:

- **Colaboração:** refere-se ao trabalho conjunto entre pessoas com o objetivo de atingir um resultado comum, por meio da troca de conhecimento, apoio mútuo e contribuições compartilhadas.

No contexto de DevOps, a interação contínua entre as equipes de desenvolvimento e operações é um dos elementos mais importantes.

- **Afinidade:** diz respeito à construção de vínculos entre equipes, buscando alinhar diferentes metas, indicadores e responsabilidades. Esse processo favorece a empatia, o entendimento mútuo e o aprendizado contínuo, sempre orientado pelos objetivos globais da organização.
- **Ferramentas:** desempenham um papel importante na aceleração de mudanças, mas seu uso deve ser criterioso. É fundamental avaliar como elas se integram ao ambiente existente, pois escolhas inadequadas podem mascarar problemas ou dificultar a colaboração. Além disso, investir em ferramentas pouco adequadas ou deixar de adotar as necessárias pode aumentar custos e comprometer a eficiência do trabalho em equipe.
- **Escalonamento:** envolve a capacidade de adaptar as práticas de DevOps conforme a organização evolui. Isso inclui ajustar processos, cultura e ferramentas de acordo com o crescimento, amadurecimento ou até redução da empresa, garantindo que os demais pilares continuem funcionando de forma equilibrada.

Além das três maneiras, ou princípios, e dos pilares apresentados, o DevOps é orientado ainda pelo modelo CALMS, que reúne cinco dimensões essenciais para sua implementação: cultura, automação, práticas enxutas (lean), medição e compartilhamento (sharing, em inglês).

A cultura é a base de tudo, pois envolve a construção de um ambiente colaborativo entre desenvolvimento, operações e demais áreas. Isso implica mudanças no comportamento organizacional, incentivando comunicação aberta, responsabilidade compartilhada e alinhamento em torno de objetivos comuns.

A automação, por sua vez, contribui para aumentar a eficiência ao reduzir tarefas manuais e repetitivas. Com isso, é possível acelerar a entrega de software, diminuir erros humanos e garantir maior consistência nos processos, liberando as equipes para atividades mais estratégicas.

O conceito lean, aplicado ao DevOps, tem como foco a eliminação de desperdícios e a entrega rápida de valor. Ele permite identificar gargalos, otimizar o uso de recursos e melhorar o fluxo de trabalho. Práticas como visualização das tarefas em andamento, limitação do trabalho em progresso e controle de filas são fundamentais nesse contexto.

A medição destaca a importância do uso de dados para orientar decisões. É fundamental observar e analisar o desempenho do sistema e dos processos para detectar gargalos e agir proativamente. Monitorar indicadores como tempo de entrega, frequência de deploys e tempo de recuperação de falhas permite avaliar o desempenho dos processos e identificar oportunidades de melhoria contínua.

O compartilhamento de conhecimento reforça a ideia de colaboração, incentivando a troca de informações, experiências e responsabilidades entre as equipes. Isso promove uma mentalidade

colaborativa, em que as equipes podem compartilhar lições aprendidas, melhores práticas, sucessos e falhas. Além disso, esse princípio fortalece o aprendizado coletivo, melhora a resolução de problemas e contribui para um ambiente mais inovador e eficiente.

Importante ressaltar que esses cinco elementos do modelo CALMS não atuam de forma isolada, mas sim de maneira integrada. A combinação entre cultura, automação, práticas enxutas, medição e compartilhamento potencializa os resultados do DevOps, criando um ciclo contínuo de melhoria, colaboração e entrega de valor.

Da mesma forma, os princípios do lean, aliados à medição orientada por dados, reconhecem que falhas são inevitáveis e, a partir disso, orientam as equipes na busca por melhoria contínua e rápida recuperação, contribuindo para a eficiência e a evolução do ciclo de desenvolvimento de software.

Portanto, a metodologia DevOps tem como propósito otimizar o ciclo de vida do desenvolvimento de software por meio da integração entre práticas, pessoas e ferramentas. Ela organiza o trabalho em fases iterativas e automatizadas, conhecidas como pipeline de CD, que viabilizam entregas rápidas e com maior qualidade. Esse ciclo ocorre de forma contínua e envolve etapas que vão desde o planejamento e desenvolvimento até a operação.

A figura 40 ilustra o ciclo de vida do DevOps e seus principais processos: planejamento (plan), codificação (code), construção (build), teste (test), lançamento (release), implantação (deploy), operação (operate) e monitoramento (monitor).

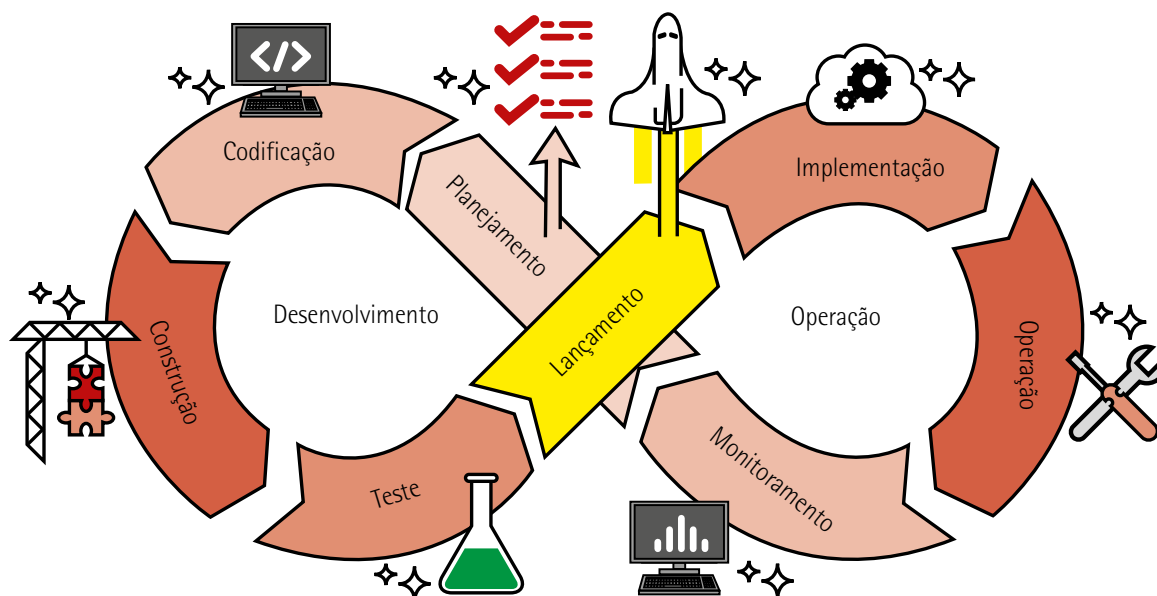


Figura 40 – Ciclo de vida do DevOps

Fonte: Mignon (2025, p. 10).

Vejam os detalhes de cada um desses processos:

- **Planejamento:** ajuda a identificar as necessidades da empresa e os valores que o projeto deve entregar. Envolve a definição dos requisitos e objetivos do projeto, determinando quais funcionalidades ou correções precisam ser desenvolvidas. Nesta fase, as equipes trabalham em conjunto para definir o escopo, criar uma lista de tarefas (backlog) e estabelecer a prioridade de cada uma.
- **Codificação:** envolve a geração de código e o design de software. O uso de boas práticas, como controle de versão e desenvolvimento ágil, é fundamental para garantir que o código seja gerenciado de forma eficiente e colaborativa.
- **Construção:** reúne todas as partes do código e as combina em uma versão funcional do software. Isso inclui compilar o código, integrar bibliotecas, frameworks e outras dependências, além de gerar os artefatos de software que serão testados.
- **Teste:** através de testes, garantimos a qualidade do software. Testes automatizados, como testes unitários, testes de integração e testes de interface, devem ser realizados para identificar os resultados antes do lançamento de uma nova versão do software.
- **Lançamento:** após a validação nos testes, o software entra na fase de lançamento. Aqui, ele é preparado, aprovado e liberado para os ambientes de produção ou pré-produção.
- **Implementação:** o software é implantado em ambiente de produção. Ferramentas de implantação automatizam o processo de entrega, garantindo que o software seja implantado de maneira eficiente e com o mínimo de intervenção manual. Dependendo da estratégia adotada, a implantação pode ser gradual ou feita diretamente em produção.
- **Operação:** após o software ser implantado, ele passa a operar em um ambiente de produção. Nesta fase, as equipes de operações monitoram o funcionamento do software para garantir que o mesmo tenha sido entregue como planejado e que funcione como esperado.
- **Monitoramento:** permite que as equipes acompanhem o desempenho do sistema, identifiquem problemas e respondam rapidamente a incidentes. Ferramentas de monitoramento são usadas para coletar métricas de uso, logs e alertas, permitindo uma análise detalhada e em tempo real. Além disso, essas métricas possibilitam melhorias contínuas, baseadas no feedback do uso em produção.

Ao integrar princípios, pilares e processos, o DevOps se consolida como uma abordagem completa para a entrega de software, promovendo automação e agilidade, para além da colaboração contínua entre equipes. Seus princípios orientam a cultura e a forma de pensar o desenvolvimento, seus pilares sustentam a estrutura organizacional e tecnológica, e o ciclo de vida operacionaliza essas ideias em práticas concretas. Como resultado, as organizações conseguem reduzir o tempo de entrega, aumentar a qualidade dos produtos e responder de maneira mais eficiente às mudanças do mercado, estabelecendo um fluxo contínuo de valor que conecta desenvolvimento e operações de forma estratégica.

7.3 Ferramentas e práticas essenciais

Explorar o universo DevOps pode ser desafiador, pois envolve diversas tecnologias, ferramentas e práticas. A gestão de múltiplos serviços em nuvem, a automação de pipelines de CI e CD, o monitoramento de aplicações distribuídas e a garantia de segurança e desempenho exigem planejamento cuidadoso e escolhas tecnológicas adequadas.

Ferramentas e práticas, como containerização com docker, orquestração com kubernetes, IaC, monitoramento contínuo, logging centralizado e rastreamento de requisições, são fundamentais para garantir a eficiência e a observabilidade dos sistemas. Além disso, a adoção de pipelines automatizados permite reduzir erros humanos e aumentar a confiabilidade das entregas.

Importante destacar que não existe uma solução única que funcione para todos os cenários. Cada organização possui necessidades específicas, o que exige adaptação constante, testes frequentes e evolução contínua dos processos e arquiteturas adotadas.

7.3.1 Integração contínua (CI)

A CI é uma prática essencial no contexto de DevOps que tem como objetivo garantir que todas as alterações realizadas no código sejam frequentemente integradas a um repositório central. Cada vez que um desenvolvedor realiza um commit, um processo automatizado é acionado para compilar o código e executar testes, validando se aquela alteração não compromete o funcionamento do sistema. Esse processo reduz significativamente os riscos associados à integração tardia de código, um problema comum em abordagens tradicionais de desenvolvimento.

Um dos principais benefícios da CI é a detecção precoce de erros. Ao integrar pequenas alterações de forma contínua, torna-se mais fácil identificar a origem de falhas, o que reduz o tempo necessário para correção. Isso evita que problemas se acumulem ao longo do tempo e se tornem mais complexos de resolver. Além disso, a prática incentiva os desenvolvedores a trabalharem com mudanças menores e mais frequentes, o que melhora a qualidade geral do código.

Outro ponto importante é a automação de testes. A CI depende fortemente de testes automatizados (unitários, de integração e, em alguns casos, funcionais) para validar o comportamento do sistema. Quanto maior a cobertura de testes, maior a confiança nas integrações realizadas, o que acaba contribuindo para um ciclo de desenvolvimento mais seguro e previsível.

Por fim, a CI promove maior colaboração entre os membros da equipe. Como todos trabalham em uma base de código compartilhada e validada constantemente, há maior transparência no progresso do projeto. Ferramentas facilitam esse processo ao integrar repositórios, pipelines e relatórios, permitindo um acompanhamento contínuo da saúde do código.

7.3.2 Entrega contínua (CD)

A CD é uma evolução direta da CI e tem como objetivo garantir que o software esteja sempre em um estado pronto para ser implantado em produção. Após passar pelas etapas de build e testes da CI, o código segue por um pipeline automatizado que valida sua qualidade e prepara sua entrega em diferentes ambientes, como desenvolvimento, homologação e produção.

Diferentemente da implantação contínua, na CD pode existir uma etapa manual de aprovação antes da liberação em produção. Essa prática é comum em organizações que desejam manter um controle adicional sobre o momento da implantação, seja por questões de negócio, disponibilidade da equipe ou janelas de manutenção. Ainda assim, todo o processo técnico anterior é automatizado, garantindo consistência e confiabilidade.

A CD também permite reduzir o tempo entre o desenvolvimento de uma funcionalidade e sua disponibilização para o usuário final. Isso aumenta a capacidade de resposta às demandas do mercado e melhora a competitividade da organização. Além disso, ao realizar entregas frequentes e menores, o risco associado a cada implantação é significativamente reduzido.

Outro aspecto relevante é a padronização do processo de entrega. Com pipelines bem definidos, todas as etapas, desde testes até a implantação, seguem um fluxo previsível e auditável. Isso facilita a identificação de falhas, a repetição de processos e a melhoria contínua do pipeline ao longo do tempo.



Observação

As siglas CI e CD com frequência aparecem escritas juntas (dessa forma: CI/CD), pois os dois conceitos formam um fluxo único e ininterrupto, também conhecido como "pipeline". Eles são as duas metades de um mesmo processo de automação. O CD não existe sem o CI. Para entregar algo continuamente (CD), é preciso primeiro integrar e testar continuamente (CI). O CI cuida da entrada do código; o CD cuida da saída para o usuário.

7.3.3 Pipelines automatizados

São o coração das práticas de CI e CD, sendo responsáveis por orquestrar todas as etapas do processo de entrega de software. Um pipeline começa com o código no repositório e segue por diversas fases, como compilação, testes, publicação de artefatos e implantação em ambientes. Esse fluxo é definido de forma automatizada, garantindo consistência e repetibilidade.

No Azure DevOps, por exemplo, os pipelines são geralmente definidos por meio de arquivos YAML, que descrevem todas as etapas do processo. Esses arquivos permitem configurar gatilhos automáticos (triggers), estágios (stages), trabalhos (jobs) e etapas (steps), que são executados por agentes responsáveis pelo processamento. Essa abordagem baseada em código facilita a manutenção e o versionamento dos pipelines.

A seguir vemos cada um desses conceitos:

- **Gatilhos automáticos:** definem quando o pipeline será executado. Podem ser acionados, por exemplo, a cada commit em um repositório, em pull requests ou em horários específicos (agendamento). São essenciais para viabilizar a CI.
- **Estágios:** representam grandes fases do pipeline, como build, teste e deploy. Cada estágio agrupa um conjunto de atividades relacionadas e normalmente segue uma sequência lógica no fluxo de entrega.
- **Trabalhos:** são conjuntos de tarefas executadas dentro de um estágio, geralmente em um mesmo agente (máquina virtual ou contêiner). Jobs podem rodar em paralelo ou em sequência, dependendo da configuração.
- **Etapas:** são as ações individuais dentro de um job, como executar um script, compilar código, rodar testes ou publicar artefatos. É o nível mais granular do pipeline.

A automação proporcionada pelos pipelines reduz significativamente a intervenção manual, minimizando erros humanos e aumentando a eficiência. Além disso, permite que as equipes tenham feedback rápido sobre o estado do código, já que cada execução do pipeline gera resultados que indicam sucesso ou falha nas etapas.

Outro benefício importante é a visibilidade do processo. Com pipelines automatizados, é possível acompanhar em tempo real o andamento das execuções, identificar gargalos e melhorar continuamente o fluxo. Isso torna o processo de entrega mais transparente e confiável, alinhado aos princípios do DevOps.

7.3.4 Integração e entrega contínua (CI e CD) com docker e azure kubernetes service

A CI e a CD pode ser implementada utilizando ferramentas como Azure DevOps, docker e azure kubernetes Service. Nesse contexto, o processo é estruturado a partir de um pipeline automatizado que gerencia desde a integração do código até sua implantação em ambientes de teste e produção (Muniz *et al.*, 2021).

O fluxo inicia com o desenvolvimento em branches, seguido pela criação de pull requests e validação do código. Após o merge na branch principal, o pipeline de CI é acionado automaticamente, realizando etapas como clonagem do repositório, compilação da aplicação, execução de análises estáticas (com ferramentas como SonarCloud) e geração de artefatos.

Em seguida, é criada uma imagem de contêiner da aplicação, que é armazenada em um repositório como o docker hub. Também são gerados arquivos de configuração (manifests) que descrevem como a aplicação será implantada no ambiente kubernetes. Esses artefatos são então utilizados na etapa de CD.

A fase de release realiza a implantação automatizada no cluster kubernetes, geralmente organizado em diferentes ambientes, como teste e produção. Esses ambientes podem ser isolados por meio de namespaces, permitindo melhor controle e organização. Em muitos casos, a promoção para produção depende de uma etapa de aprovação, garantindo maior governança no processo.

Por fim, todo o pipeline é definido como código (YAML), o que permite versionamento, reutilização e padronização do processo. Essa abordagem reforça os princípios do DevOps, promovendo automação, rastreabilidade e maior confiabilidade na entrega de software.

7.3.5 Infraestrutura como código (IaC)

A IaC consiste na capacidade de provisionar e gerenciar recursos computacionais por meio de código, substituindo configurações manuais e processos operacionais tradicionais. Todo ambiente de software depende de diversos componentes, como sistemas operacionais, bancos de dados, redes e armazenamento, que precisam ser configurados e mantidos continuamente para suportar o desenvolvimento, testes e implantação de aplicações. Quando esse gerenciamento é feito manualmente, tende a ser lento, repetitivo e sujeito a falhas, especialmente em cenários de grande escala.

A IaC representa uma mudança significativa na forma como os ambientes de TI são provisionados e gerenciados. Em vez de configurar servidores e recursos manualmente, toda a infraestrutura passa a ser definida por meio de código, que pode ser versionado, testado e reutilizado. Isso aproxima a gestão de infraestrutura das práticas já consolidadas no desenvolvimento de software.

Com a IaC, é possível declarar o estado desejado do ambiente de forma automatizada, sem a necessidade de executar passo a passo cada configuração. Isso permite padronizar processos, reduzir erros e liberar os desenvolvedores para focarem na criação e evolução das aplicações, em vez de tarefas operacionais. Além disso, essa abordagem contribui para maior controle sobre os recursos utilizados, ajudando na redução de custos, no aumento da confiabilidade e na capacidade de resposta rápida às demandas do negócio.

Um dos principais benefícios da IaC é a consistência entre ambientes. Como a infraestrutura é definida por código, é possível reproduzir exatamente o mesmo ambiente em diferentes estágios, como desenvolvimento, teste e produção. Isso reduz problemas relacionados a diferenças de configuração, conhecidos como "funciona na minha máquina".

A IaC permite maior automação e escalabilidade. Recursos como VMs, redes e bancos de dados podem ser criados ou removidos automaticamente conforme a necessidade, o que é especialmente útil em ambientes de nuvem. Ferramentas DevOps permitem incluir a criação da infraestrutura diretamente nos pipelines, tornando o processo ainda mais eficiente. Outro ponto importante é a rastreabilidade. Como as configurações estão versionadas, é possível saber exatamente quais mudanças foram feitas, quando e por quem. Isso facilita auditorias, controle de mudanças e recuperação de ambientes em caso de falhas, contribuindo para uma operação mais segura e confiável.

7.3.6 Gestão de configuração

É uma prática essencial para assegurar que todos os componentes necessários ao funcionamento de um sistema estejam organizados, versionados e sob controle. Isso abrange não apenas o código-fonte, mas também variáveis de ambiente, dependências, parâmetros de execução e definições de infraestrutura. Ao tratar esses elementos de forma estruturada, evita-se a inconsistência entre ambientes e reduz-se o risco de falhas decorrentes de configurações incorretas ou divergentes.

No contexto de DevOps, a gestão de configuração desempenha um papel central na garantia de consistência entre os diferentes ambientes utilizados ao longo do ciclo de vida do software, como desenvolvimento, teste e produção. Ao padronizar e centralizar as configurações, torna-se possível reproduzir ambientes de forma previsível, reduzindo problemas comuns como “funciona na minha máquina”. Essa consistência é fundamental para aumentar a confiabilidade das entregas e a estabilidade das aplicações em produção.

Ganha destaque a reutilização de configurações. Elementos comuns podem ser compartilhados entre diferentes aplicações, serviços e pipelines, evitando duplicidade e simplificando a manutenção. Essa prática não apenas reduz o esforço operacional, como também contribui para a padronização dos processos, facilitando a adoção de boas práticas e a escalabilidade das soluções dentro da organização.

Além disso, a gestão de configuração está diretamente relacionada à rastreabilidade e à governança. Manter o histórico das configurações utilizadas em cada versão ou implantação permite auditar mudanças, identificar rapidamente a origem de falhas e garantir conformidade com normas e políticas internas. Esse nível de controle é especialmente importante em ambientes críticos, onde segurança e conformidade são requisitos fundamentais.

Outro aspecto importante é a integração da gestão de configuração com práticas como IaC e pipelines de CI e CD. Ao versionar configurações junto com o código e automatizar sua aplicação, as equipes conseguem garantir que qualquer alteração seja devidamente testada e validada antes de chegar à produção. Isso reduz significativamente o risco de erros humanos e aumenta a previsibilidade do processo de entrega.

A gestão de configuração também contribui para a escalabilidade e resiliência dos sistemas. Em ambientes distribuídos e dinâmicos, onde novos recursos podem ser criados ou removidos rapidamente, ter configurações bem definidas e automatizadas permite adaptar a infraestrutura de forma ágil e segura. Dessa forma, a organização consegue responder mais rapidamente às mudanças do negócio, mantendo a qualidade e a estabilidade das aplicações.

7.4 Integração entre desenvolvimento e operações

Vimos que as metodologias de desenvolvimento de software passaram por diversas transformações ao longo do tempo, mas o processo de converter ideias em soluções concretas ainda envolve etapas fundamentais, como levantamento de requisitos, definição de design, elaboração da arquitetura, implementação e realização de testes. Nesse cenário, os métodos ágeis surgiram como uma alternativa ao

modelo tradicional, propondo uma forma mais dinâmica de organizar essas atividades. Diferentemente do modelo em cascata, no qual cada fase ocorre de maneira sequencial, os métodos ágeis promovem a execução simultânea dessas etapas por meio de ciclos curtos e contínuos, conhecidos como iterações. Ao término de cada ciclo, o software evolui gradualmente, tornando-se mais funcional, com a adição de novas características e a redução de falhas, enquanto a equipe, em conjunto com o cliente, define os próximos incrementos a serem desenvolvidos.

Quando o cliente considera que o sistema está pronto e o software é disponibilizado em ambiente de produção, inicia-se uma nova etapa, na qual os usuários finais passam a utilizá-lo de fato. A partir desse momento, surgem outras preocupações igualmente importantes, como suporte técnico, monitoramento constante, segurança da informação, garantia de disponibilidade, desempenho e experiência do usuário. Em produção, a principal prioridade passa a ser a manutenção da estabilidade do sistema, assegurando que ele continue funcionando corretamente. Em situações de falhas ou incidentes, a equipe responsável deve agir de forma rápida e eficiente, adotando uma postura proativa para identificar e resolver problemas, minimizando impactos para os usuários e para o negócio.

Devido às características distintas dessas atividades, é comum que departamentos de TI estabeleçam uma separação clara entre as responsabilidades das equipes de desenvolvimento e de operações. Enquanto o time de desenvolvimento se concentra na criação de novas aplicações, na implementação de funcionalidades e na correção de erros, o time de operações dedica-se à gestão e manutenção dos sistemas já em funcionamento. Dessa forma, cada equipe atua com objetivos específicos: o desenvolvimento busca inovação e evolução contínua do software, enquanto operações prioriza a confiabilidade e a estabilidade dos serviços em produção.

À primeira vista, essa divisão de responsabilidades parece lógica e eficiente, uma vez que cada equipe possui competências, ferramentas e formas de trabalho distintas. O time de desenvolvimento costuma atuar de maneira iterativa, planejando e entregando melhorias progressivas, enquanto o time de operações precisa responder rapidamente a incidentes e garantir o funcionamento ininterrupto dos sistemas. Além disso, o conhecimento técnico exigido em cada área tende a ser especializado. Enquanto os desenvolvedores promovem mudanças e evoluções no sistema, a equipe de operações atua no sentido de preservar sua estabilidade e continuidade, o que pode gerar, em alguns casos, conflitos de interesse entre inovação e confiabilidade.

O DevOps assume um papel central ao promover a integração entre as equipes de desenvolvimento e operações, eliminando barreiras tradicionais e incentivando uma cultura de colaboração contínua. Essa abordagem possibilita a automação de etapas críticas, como integração, testes e entrega de software, por meio de pipelines de CI/CD, reduzindo erros manuais e aumentando a confiabilidade das entregas. Além disso, ao utilizar recursos nativos da nuvem, como escalabilidade sob demanda, monitoramento em tempo real e provisionamento automatizado de infraestrutura, as organizações conseguem implementar práticas mais eficientes e resilientes.

Em um cenário cada vez mais digital, o DevOps tem papel central na sustentação de serviços essenciais, como streaming de mídia, comércio eletrônico, bancos digitais, redes sociais e aplicações de IoT, que exigem alta disponibilidade, desempenho e atualizações constantes.



Lembrete

O objetivo do DevOps é tornar os processos mais simples, integrar desenvolvedores e profissionais de operações, unir as equipes e aumentar as entregas aos clientes, agregando velocidade e qualidade a elas.

O DevOps pode ser compreendido como uma cultura aliada a um conjunto de práticas que integram as equipes de desenvolvimento e operações, promovendo colaboração contínua e automação ao longo de todo o ciclo de vida do software. Essa abordagem permite tornar os processos mais ágeis, desde a codificação até a entrega e manutenção das aplicações em produção.

Entre os principais benefícios do DevOps estão a aceleração na entrega de novas funcionalidades, a melhoria na qualidade do software, a redução de falhas em produção e a maior eficiência operacional. Além disso, a automação de processos, como CI e CD, possibilita respostas mais rápidas às mudanças do mercado e às necessidades dos usuários, aumentando a competitividade das organizações.

A figura a seguir ilustra o ciclo contínuo do DevOps, evidenciando a integração entre as etapas de desenvolvimento (application design and development), implantação (deployment) e manutenção de aplicações (maintenance). A primeira está relacionada à concepção e construção do software, envolvendo desde o design até a implementação e testes iniciais. A segunda trata da disponibilização da aplicação nos ambientes de execução, com foco em automação, CI e entrega ágil. Já a terceira refere-se à operação e sustentação do sistema em produção, abrangendo monitoramento, análise de desempenho e gestão de recursos. Em conjunto, essas áreas representam um fluxo contínuo e integrado, no qual o software é constantemente desenvolvido, entregue, monitorado e aprimorado.

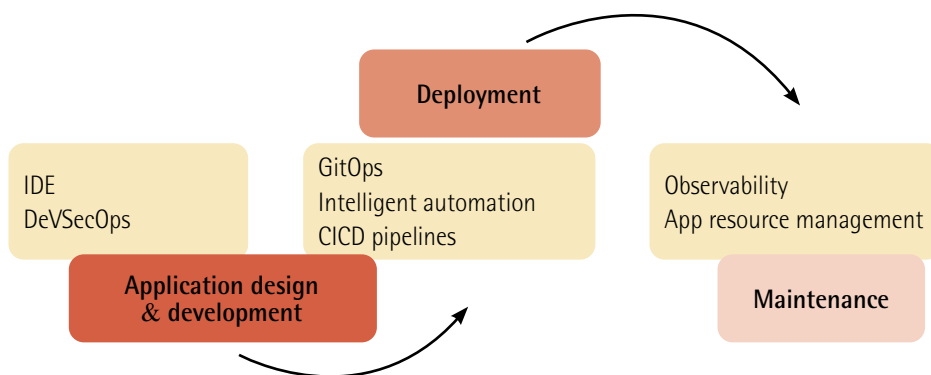


Figura 41 – Ciclo de vida e práticas DevOps (imagem produzida com a ferramenta de IA generativa ChatGPT, da OpenAI)

Esse fluxo demonstra como práticas, ferramentas e processos atuam de forma conjunta para garantir entregas rápidas, seguras e com qualidade, reforçando a ideia de que o software está em constante evolução. Além disso, destaca o papel da automação, da observabilidade e da colaboração entre equipes ao longo de todo o ciclo de vida da aplicação.

O uso de integrated development environments (IDEs) é fundamental na etapa de desenvolvimento, pois oferece aos programadores um ambiente completo para escrita, teste e depuração de código. Essas ferramentas aumentam a produtividade e reduzem erros, facilitando a construção de aplicações mais robustas e organizadas. Além disso, as IDEs modernas frequentemente se integram com repositórios de código e pipelines automatizados, conectando o desenvolvimento às etapas seguintes do ciclo DevOps.

Outro aspecto relevante nessa fase é o DevSecOps, que incorpora práticas de segurança diretamente no processo de desenvolvimento. Em vez de tratar a segurança como uma etapa posterior, ela passa a ser considerada desde o início, com a realização de análises de vulnerabilidades, revisão de código e testes automatizados. Isso contribui para a construção de aplicações mais seguras e reduz riscos ao longo do ciclo de vida.

Na etapa de deployment, ferramentas e práticas como GitOps desempenham papel importante ao utilizar repositórios Git como fonte única de verdade para a infraestrutura e as aplicações. Isso permite maior controle, rastreabilidade e automação no processo de implantação, garantindo consistência entre os ambientes.

A automação inteligente também se destaca nesse contexto, permitindo a execução automatizada de tarefas complexas, como provisionamento de infraestrutura, testes e deploys. Aliado a isso, os pipelines de CI/CD viabilizam a integração frequente de código e sua rápida disponibilização em produção. Essas práticas reduzem o tempo de entrega, aumentam a confiabilidade e minimizam falhas humanas.

Na fase de manutenção, a observabilidade é essencial para garantir o bom funcionamento das aplicações em produção. Por meio do monitoramento contínuo, coleta de métricas, logs e rastreamento de eventos, é possível identificar problemas rapidamente e agir de forma proativa, evitando impactos maiores aos usuários.

Além disso, o gerenciamento de recursos da aplicação (app resource management) permite acompanhar e otimizar o uso de infraestrutura, garantindo desempenho adequado e controle de custos. Em ambientes de nuvem, onde os recursos são escaláveis e cobrados sob demanda, essa prática se torna ainda mais importante para manter a eficiência operacional e financeira.

8 INOVAÇÃO E FUTURO DA COMPUTAÇÃO EM NUVEM E DEVOPS

As aplicações modernas estão cada vez mais apoiadas na computação em nuvem e nas práticas de DevOps, integrando desde servidores de alto desempenho até dispositivos móveis, todos conectados e colaborando para entregar serviços digitais de forma contínua e eficiente. A colaboração entre diferentes componentes e equipes torna-se essencial para garantir agilidade e qualidade na entrega de software.

A combinação entre computação em nuvem e DevOps oferece um enorme potencial de transformação. Quando bem aplicadas, essas abordagens permitem inovação acelerada, maior resiliência dos serviços, escalabilidade sob demanda e uma vantagem competitiva significativa no mercado.

O sucesso nesse contexto depende não apenas da adoção de tecnologias modernas, mas também da capacidade de implementar boas práticas, automatizar processos, monitorar continuamente os sistemas e aprender com os erros. Dominar computação em nuvem e DevOps tornou-se, portanto, um fator estratégico para organizações que buscam crescimento sustentável e inovação constante.

Diante dessa relevância, a seguir serão apresentadas algumas tendências e ferramentas amplamente utilizadas nesse contexto.

8.1 Integração com IA

A integração entre computação em nuvem e IA é uma das tendências mais relevantes do cenário tecnológico atual. A nuvem fornece a infraestrutura necessária para o treinamento e a execução de modelos de aprendizado de máquina em larga escala, oferecendo recursos computacionais sob demanda. Essa combinação permite que organizações desenvolvam soluções inteligentes sem a necessidade de investir em infraestrutura própria, reduzindo custos e acelerando a inovação.

A IA pode ser entendida como a capacidade de sistemas computacionais compostos por software, hardware e sensores de simular funções cognitivas humanas com o objetivo de resolver problemas por meio do aprendizado (Silva *et al.*, 2023). Trata-se de um campo abrangente, que engloba diversas técnicas voltadas à reprodução de comportamentos considerados inteligentes, como tomada de decisão, planejamento, reconhecimento de padrões, raciocínio lógico, sistemas especialistas, busca heurística e aprendizado de máquina.

Primeiramente, é importante diferenciar IA, machine learning (ML) e deep learning (DL). A IA corresponde ao campo mais amplo, que reúne diferentes abordagens para construção de sistemas inteligentes. O machine learning é um subconjunto da IA, baseado na capacidade dos sistemas de aprenderem a partir de dados, sem depender exclusivamente de regras previamente definidas. Já o deep learning é uma especialização do machine learning que utiliza redes neurais profundas, sendo particularmente eficaz em problemas complexos, como reconhecimento de imagens, processamento de linguagem natural e reconhecimento de fala. Assim, todo deep learning é machine learning e também IA, mas nem toda IA utiliza aprendizado de máquina.

A figura a seguir demonstra essa relação hierárquica que existe entre IA, ML e DL, indicando que esses conceitos não são sinônimos, mas sim níveis diferentes dentro de um mesmo campo.

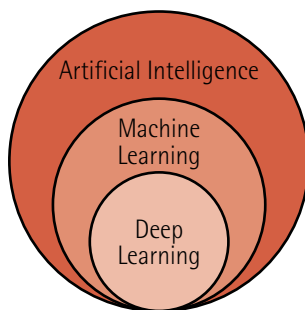


Figura 42 – Hierarquia da IA

Disponível em: <https://tinyurl.com/5y368rc5>. Acesso em: 1 jun. 2026.

A disponibilização de serviços de IA em nuvem ampliou significativamente o acesso a essas tecnologias. Provedores oferecem modelos prontos, APIs e plataformas completas que permitem incorporar funcionalidades inteligentes em aplicações de forma simplificada, reduzindo barreiras técnicas e financeiras. Esse movimento tem alavancado a adoção da IA em diferentes setores e contextos organizacionais.

O crescimento exponencial da geração de dados, impulsionado por dispositivos móveis, redes sociais, sensores IoT e sistemas corporativos, exige ambientes capazes de processar grandes volumes de informação. A computação em nuvem atende a essa demanda ao disponibilizar recursos escaláveis, como clusters de alto desempenho, viabilizando o treinamento de modelos complexos, especialmente em cenários de aprendizado profundo.

É importante destacar a diferença entre dados e informação. Dados são registros brutos, enquanto a informação resulta da organização, processamento e interpretação desses dados, permitindo seu uso estratégico. Nesse contexto, tecnologias como big data e IA desempenham papel essencial ao possibilitar a análise de grandes volumes de dados, caracterizados por sua variedade, velocidade e volume. A qualidade desses dados é determinante, pois decisões eficazes dependem não apenas da quantidade, mas também da confiabilidade das informações.

O uso de ferramentas computacionais para análise de dados amplia a capacidade analítica dos profissionais, permitindo a identificação de padrões e a geração de insights que dificilmente seriam obtidos por métodos tradicionais. No mercado, por exemplo, a análise de comportamento do consumidor possibilita estratégias mais direcionadas, como recomendações personalizadas e campanhas de marketing em tempo real.

Outra tendência importante é o modelo machine learning as a service (MLaaS), no qual provedores de nuvem oferecem plataformas completas para todo o ciclo de vida do aprendizado de máquina, incluindo preparação de dados, treinamento, validação e implantação. Essa abordagem facilita a adoção da IA, permitindo que organizações integrem essas soluções com maior agilidade.

A IA generativa também vem ganhando destaque, permitindo a criação automática de conteúdos como textos, imagens e códigos. Esse tipo de aplicação depende fortemente de infraestruturas distribuídas e de alto desempenho, fornecidas pela nuvem, que viabilizam tanto o treinamento quanto a utilização desses modelos em larga escala.

Paralelamente, observa-se a consolidação do conceito de MLOps, que integra práticas de DevOps ao ciclo de vida dos modelos de aprendizado de máquina. Nesse cenário, a computação em nuvem desempenha um papel fundamental ao viabilizar a automação de pipelines de treinamento, o versionamento de modelos e o monitoramento contínuo de desempenho.

Além disso, a integração entre IA e DevOps vem evoluindo para além do MLOps, incorporando o uso de algoritmos inteligentes para otimizar processos de desenvolvimento e operação, como detecção automática de falhas, análise preditiva de incidentes e automação de testes, fortalecendo a eficiência e a confiabilidade dos sistemas.

A convergência entre nuvem e IA estabelece uma relação complementar: enquanto a nuvem fornece capacidade de armazenamento e processamento em escala, a IA transforma dados em conhecimento, automatiza processos e apoia a tomada de decisão. Essa sinergia amplia o potencial das organizações em diferentes setores.

As aplicações de IA são amplas e abrangem áreas como serviços, com o uso de chatbots e sistemas de atendimento automatizado; ambientes domésticos, com assistentes virtuais e automação residencial; cidades inteligentes, com gestão de tráfego e infraestrutura urbana; indústria 4.0, com manutenção preditiva e automação de processos; agricultura de precisão, com análise de dados e monitoramento de cultivos; e saúde, com apoio ao diagnóstico e à gestão de atendimentos.

Além do nível operacional, a IA agrega valor estratégico ao apoiar a tomada de decisão, melhorar o engajamento de clientes e colaboradores e promover a automação de processos. Sua aplicação contribui para maior agilidade, personalização de serviços e ganhos de eficiência organizacional.

A própria infraestrutura em nuvem pode ser otimizada com o uso de IA, por meio da previsão de falhas, alocação inteligente de recursos e ajuste dinâmico de cargas de trabalho. A integração com edge computing amplia ainda mais esse cenário, permitindo que modelos treinados na nuvem sejam executados próximos à origem dos dados, reduzindo latência e viabilizando decisões em tempo real. Adicionalmente, a combinação de microsserviços com IA possibilita a criação de sistemas mais autônomos, capazes de adaptar seu comportamento com base em padrões de uso. Arquiteturas orientadas a eventos (event-driven) também ganham destaque, permitindo que sistemas reajam automaticamente a eventos ou gatilhos específicos.

No campo da segurança, a IA contribui para a detecção de ameaças cibernéticas ao identificar padrões anômalos em grandes volumes de dados, aumentando a proteção dos ambientes digitais. A observabilidade torna-se um elemento crítico, exigindo ferramentas avançadas de monitoramento, rastreamento distribuído e análise de logs para garantir confiabilidade. A segurança também evolui, demandando abordagens como autenticação descentralizada e políticas de acesso baseadas em identidade e contexto.



Observação

A IA capacita os sistemas de segurança cibernética a analisar grandes quantidades de dados, identificar padrões e tomar decisões informadas, em velocidades e escalas além das capacidades humanas (O que..., s.d.).

Como tendência futura, observa-se um aumento no nível de abstração da infraestrutura, com plataformas cada vez mais gerenciadas e integradas nativamente a recursos de IA. Esse movimento aponta para a consolidação de ambientes altamente automatizados, nos quais a convergência entre computação em nuvem, IA e práticas de DevOps desempenha papel central na transformação digital das organizações.

8.2 Computação quântica

A computação quântica desponta como uma das tendências mais promissoras no cenário da computação em nuvem e do DevOps, ao propor um novo paradigma de processamento baseado nos princípios da mecânica quântica. Diferentemente dos computadores clássicos, que utilizam bits representados por 0 ou 1, os computadores quânticos operam com qubits, que podem assumir múltiplos estados simultaneamente por meio do fenômeno da superposição. Além disso, o entrelaçamento quântico permite que qubits estejam correlacionados de forma que a alteração em um afete diretamente o outro, ampliando significativamente a capacidade de processamento para determinados tipos de problema.

Embora ainda esteja em estágio inicial de maturidade, a computação quântica já apresenta potencial para impactar áreas que demandam alto poder computacional, como criptografia, otimização de processos, simulação de moléculas e desenvolvimento de novos materiais.

Problemas que seriam inviáveis de resolver com computadores tradicionais, mesmo usando grandes clusters em nuvem, podem se tornar tratáveis com algoritmos quânticos, abrindo novas possibilidades para a ciência e para a indústria.

No contexto da computação em nuvem, observa-se o surgimento do modelo quantum computing as a service (QCaaS), no qual provedores passam a disponibilizar acesso remoto a computadores quânticos por meio da nuvem. Essa abordagem democratiza o uso da tecnologia, permitindo que pesquisadores, empresas e desenvolvedores experimentem algoritmos quânticos sem a necessidade de investir em infraestrutura altamente especializada e ainda extremamente custosa.

A integração entre computação quântica e práticas de desenvolvimento também começa a ganhar espaço. Embora ainda distante do cotidiano do DevOps tradicional, já se discute a adaptação de pipelines e ferramentas para suportar o desenvolvimento híbrido, combinando computação clássica e quântica. Nesse cenário, o papel do desenvolvedor tende a evoluir, exigindo novos conhecimentos relacionados a algoritmos quânticos, linguagens específicas e modelagem de problemas adequados a esse tipo de processamento.

Outro ponto relevante diz respeito à segurança da informação. A computação quântica tem o potencial de quebrar algoritmos criptográficos amplamente utilizados atualmente, o que impulsiona pesquisas em criptografia pós-quântica, voltada ao desenvolvimento de métodos resistentes a ataques realizados por computadores quânticos. Esse movimento já impacta diretamente estratégias de segurança em ambientes de nuvem.

Apesar dos avanços, ainda existem desafios significativos, como a instabilidade dos qubits, a necessidade de ambientes altamente controlados e a limitação no número de operações confiáveis. No entanto, o ritmo de evolução da área indica que a computação quântica tende a se tornar, em médio e longo prazo, um componente relevante do ecossistema tecnológico.

Assim, no contexto da computação em nuvem e do DevOps, a computação quântica pode ser compreendida como uma tendência emergente que, embora ainda em desenvolvimento, possui grande

potencial transformador. Sua integração com a nuvem amplia o acesso e acelera a experimentação, preparando o caminho para uma nova geração de soluções capazes de lidar com problemas de alta complexidade de forma mais eficiente.

8.3 Tendências em computação em nuvem

Pelas estratégias e práticas do mercado, é evidente que a computação em nuvem deixou de ser apenas uma opção tecnológica para se firmar como base essencial da transformação digital em diversos setores da sociedade. Organizações públicas, privadas e acadêmicas passaram a utilizá-la não só com o objetivo de reduzir custos ou ampliar a capacidade de armazenamento, mas principalmente para viabilizar inovação contínua, alcançar escalabilidade em nível global e sustentar modelos de negócio orientados por dados. Nesse cenário, a evolução da nuvem acompanha o surgimento de novas tecnologias, tornando-se cada vez mais integrada a ambientes complexos e distribuídos.

O futuro da computação em nuvem está fortemente ligado à sua integração com áreas como IA, à adoção de novos modelos arquiteturais como serverless e microsserviços, à expansão da IoT e à crescente preocupação com sustentabilidade e eficiência energética. Esses fatores não apenas ampliam as capacidades da nuvem, mas também reforçam seu papel estratégico na economia digital. A seguir, são apresentadas as principais tendências relacionadas a esses aspectos.

8.3.1 Serverless

A evolução arquitetural das aplicações em nuvem tem sido marcada pela transição de sistemas monolíticos para arquiteturas baseadas em microsserviços. Em sistemas distribuídos modernos, a arquitetura de microsserviços quebra uma aplicação grande em vários serviços pequenos, independentes e especializados, que se comunicam entre si, normalmente via APIs. Cada microsserviço é responsável por uma única função de negócio. Essa mudança visa aumentar flexibilidade, escalabilidade e velocidade de desenvolvimento.

Os microsserviços segmentam aplicações em componentes independentes, cada um responsável por uma funcionalidade específica. Essa abordagem facilita atualizações isoladas, reduz impacto de falhas e permite escalonamento granular. A popularização de contêineres, especialmente com docker, e de orquestradores como kubernetes fortaleceu essa tendência ao oferecer mecanismos eficientes de gerenciamento e implantação de aplicações distribuídas.

Nesse cenário, surge o paradigma serverless, no qual o desenvolvedor não gerencia servidores explicitamente. O provedor de nuvem executa funções sob demanda, escalando automaticamente conforme o volume de requisições.

Do ponto de vista arquitetural, o serverless está fortemente associado a aplicações orientadas a eventos. Isso significa que as funções são acionadas por gatilhos específicos, como uma requisição HTTP, o envio de uma mensagem para uma fila, a atualização em um banco de dados ou o upload de um arquivo em um storage. Cada função executa uma tarefa bem definida, geralmente de curta duração e sem manter estado entre execuções (stateless). Essa característica favorece arquiteturas baseadas em microsserviços e integrações desacopladas, bastante comuns em sistemas distribuídos modernos.

Além disso, o serverless transfere para o provedor de nuvem responsabilidades como provisionamento de infraestrutura, balanceamento de carga, escalabilidade automática e tolerância a falhas. Um exemplo de modelo de serviço dentro do paradigma serverless é o Function as a Service (FaaS), que permite cobrança baseada no tempo real de execução, promovendo maior eficiência financeira. Isso é especialmente vantajoso para cargas intermitentes, em que a aplicação não precisa manter recursos computacionais ativos o tempo todo, pagando apenas quando há processamento efetivo.

Exemplos de plataformas que implementam o modelo FaaS incluem o AWS Lambda, o Azure Functions e o Google Cloud Functions. É importante compreender que, embora o termo "serverless" sugira ausência de servidores, eles continuam existindo; a diferença é que sua gestão é totalmente abstraída pelo provedor.

O avanço de tecnologias como virtualização, provisionamento automatizado e arquiteturas serverless tem ampliado as possibilidades de desenvolvimento de sistemas mais escaláveis, modulares e desacoplados. Essas abordagens favorecem práticas como o desenvolvimento incremental e a reutilização de componentes, promovendo maior agilidade e alinhamento entre as soluções tecnológicas e os objetivos de negócio.

No entanto, tais avanços também trazem novos desafios, sobretudo no que diz respeito à modernização de sistemas legados e ao aumento da complexidade na segurança de ambientes distribuídos. A decomposição de aplicações em múltiplos serviços exige a adoção de mecanismos mais robustos de monitoramento, autenticação e controle de acesso, a fim de garantir a confiabilidade e a proteção das informações.

8.3.2 Expansão do edge computing

A expansão do edge computing representa uma mudança importante na forma como os sistemas computacionais são projetados e distribuídos. Tradicionalmente, o processamento de dados ocorre de forma centralizada em grandes data centers ou na nuvem. No entanto, com o crescimento exponencial de dispositivos conectados, como sensores, smartphones, veículos inteligentes e sistemas industriais, surgiu a necessidade de processar parte desses dados mais próximo da sua origem. O edge computing se desenvolve justamente para atender a essa demanda, levando o processamento para a "borda" da rede, reduzindo a dependência de servidores centralizados.

Essa descentralização traz benefícios significativos em termos de desempenho e eficiência. Ao processar dados localmente ou em nós intermediários, diminui-se a latência, ou seja, o tempo entre a geração e a resposta do sistema. Isso é especialmente crítico em aplicações que exigem resposta em tempo real, como carros autônomos, sistemas de saúde, automação industrial e cidades inteligentes. Além disso, o edge computing reduz o tráfego de dados enviado para a nuvem, o que pode gerar economia de largura de banda e melhorar a escalabilidade dos sistemas.

Outro fator que impulsiona a expansão do edge computing é o avanço da IoT. Com bilhões de dispositivos gerando dados continuamente, enviar tudo para processamento centralizado torna-se inviável em muitos cenários. Ao processar parte dessas informações na borda da rede, os sistemas conseguem filtrar, analisar e até tomar decisões localmente, enviando para a nuvem apenas dados relevantes ou agregados. Isso torna a arquitetura mais eficiente e resiliente.



Lembrete

Edge computing é um modelo de computação no qual o processamento de dados ocorre o mais próximo possível da origem da informação, como sensores IoT ou dispositivos móveis. Isso reduz a latência, desafoga a nuvem e aumenta a eficiência em aplicações que demandam respostas em tempo real, como carros autônomos, fábricas inteligentes e jogos online.

O edge computing contribui também para maior confiabilidade e continuidade dos serviços. Em ambientes onde a conexão com a nuvem pode ser instável ou limitada, como áreas remotas ou ambientes industriais críticos, a capacidade de processamento local garante que o sistema continue operando mesmo sem conectividade constante. Isso aumenta a autonomia dos dispositivos e reduz riscos de interrupção.

No entanto, a adoção do edge computing também traz desafios. A distribuição do processamento em múltiplos nós aumenta a complexidade de gerenciamento, monitoramento e segurança. Garantir a proteção dos dados em diversos pontos da rede exige novas estratégias de segurança e governança. Além disso, manter a consistência entre sistemas distribuídos pode ser mais difícil do que em arquiteturas centralizadas.

Ainda assim, a tendência é de crescimento contínuo dessa abordagem, especialmente com o avanço de tecnologias como 5G, IA e computação distribuída. Essas inovações tornam o edge computing cada vez mais viável e necessário para suportar aplicações modernas que exigem baixa latência, alta disponibilidade e processamento em tempo real. Assim, ele passa a complementar a nuvem, em vez de substituí-la.

Em síntese, a expansão do edge computing representa uma evolução arquitetural significativa na computação moderna, deslocando parte do processamento para mais próximo dos usuários e dispositivos. Essa mudança fortalece o desempenho, melhora a eficiência e habilita novas aplicações que dependem de respostas rápidas e processamento distribuído, consolidando um modelo híbrido entre borda e nuvem como padrão emergente na infraestrutura digital.



Saiba mais

Para saber mais sobre edge computing e ter acesso a estudos de caso que ilustram sua aplicação, leia:

SHI, W. et al. Edge computing: vision and challenges. *IEEE Internet of Things Journal*, v. 3, n. 5, p. 637-646, out. 2016. Disponível em: <https://tinyurl.com/mr3vfubh>. Acesso em: 20 maio 2026.

8.3.3 Green cloud e sustentabilidade

À medida que cresce a conscientização da sociedade sobre os impactos ambientais decorrentes do modelo de desenvolvimento vigente, tornam-se mais evidentes os desafios na relação entre atividade humana e preservação do meio ambiente. Esse contexto tem impulsionado reflexões e ações voltadas à construção de um modelo de desenvolvimento que concilie progresso econômico, bem-estar social e proteção ambiental, promovendo melhores condições de vida sem comprometer o futuro.

Nesse sentido, o conceito de sustentabilidade baseia-se na ideia de atender às demandas atuais sem prejudicar a capacidade das próximas gerações de suprirem suas próprias necessidades. Trata-se de uma abordagem sistêmica, que considera os efeitos de longo prazo e busca equilibrar três dimensões fundamentais: ambiental, social e econômica. Com o passar do tempo, a sustentabilidade deixou de ser um tema restrito ao meio ambiente e passou a orientar decisões estratégicas em políticas públicas, práticas empresariais e adoção de tecnologias.

A partir da década de 1990, o termo ganhou maior visibilidade, sendo amplamente associado a propostas de desenvolvimento mais responsáveis do ponto de vista ambiental. No entanto, sua popularização nem sempre foi acompanhada de uma compreensão aprofundada sobre sua aplicação prática. A complexidade do tema decorre justamente de sua natureza multidimensional, que envolve diferentes interpretações e abordagens. De modo geral, pode-se entender sustentabilidade como a busca por equilíbrio entre as dimensões ambiental, social e econômica que sustentam a sociedade (Bado; Vione, 2022).

Quando associado ao desenvolvimento econômico, o conceito de sustentabilidade evidencia desafios importantes, especialmente no que diz respeito à conciliação entre crescimento e preservação ambiental. Esse dilema revela a dificuldade de alinhar interesses imediatos com a necessidade de conservação dos recursos naturais. Embora existam iniciativas globais voltadas a esse equilíbrio, como a Agenda 2030 e os Objetivos de Desenvolvimento Sustentável (United Nations, 2024), alcançar esse objetivo exige mudanças estruturais nos modelos tradicionais de crescimento.

A sustentabilidade é tradicionalmente ancorada por três pilares interdependentes. O ambiental está relacionado à conservação dos recursos naturais, à redução da poluição, ao uso eficiente de energia e ao combate às mudanças climáticas. O social envolve a promoção da equidade, da inclusão e da qualidade de vida, garantindo acesso justo a oportunidades e serviços. Já o econômico busca assegurar crescimento sustentável e geração de valor de forma responsável, sem comprometer recursos ou ampliar desigualdades (Bado; Vione, 2022).

No quesito ambiental, destacam-se preocupações com o uso racional dos recursos e a redução das emissões de gases de efeito estufa. O aumento da demanda por energia, impulsionado pela digitalização e pela expansão das tecnologias, intensificou o debate sobre os impactos ambientais das atividades econômicas. Como resposta, práticas como eficiência energética, adoção de fontes renováveis e economia circular têm ganhado relevância.

Sob a perspectiva social, a sustentabilidade implica garantir que os benefícios do avanço tecnológico sejam amplamente distribuídos. Isso inclui reduzir desigualdades no acesso à tecnologia, investir em capacitação profissional e assegurar condições adequadas de trabalho. A transformação digital, portanto, deve ser acompanhada por iniciativas que promovam inclusão e acesso equitativo à informação.

No campo econômico, a sustentabilidade está associada à construção de modelos de negócio resilientes e inovadores, capazes de gerar valor de forma contínua. Organizações que incorporam práticas sustentáveis tendem a reduzir riscos, fortalecer sua imagem institucional e atender a um público cada vez mais consciente. Assim, sustentabilidade e competitividade passam a caminhar juntas.

A integração desses três pilares exige que decisões estratégicas considerem simultaneamente seus impactos ambientais, sociais e econômicos. No contexto tecnológico, isso significa avaliar não apenas desempenho e custo, mas também consumo de energia, ciclo de vida dos equipamentos e efeitos indiretos sobre a sociedade.

Ao relacionar sustentabilidade com computação em nuvem, surge o conceito de green cloud, que propõe alinhar eficiência tecnológica à responsabilidade ambiental. A computação em nuvem verde refere-se ao uso de arquiteturas e práticas voltadas à redução do impacto ambiental, sobretudo no que diz respeito ao consumo energético, à emissão de carbono e ao uso eficiente de recursos computacionais.

De forma geral, o conceito envolve a adoção de práticas sustentáveis no projeto, na operação e na gestão de data centers e serviços em nuvem, buscando minimizar desperdícios sem comprometer desempenho, disponibilidade ou escalabilidade. O crescimento acelerado dos data centers em escala global aumentou significativamente o consumo de energia, tornando essencial a adoção de soluções mais eficientes.

Entre as principais iniciativas, destacam-se o uso de fontes de energia renovável, como solar e eólica, e o compromisso de grandes provedores com metas de neutralidade de carbono. Além disso, técnicas de virtualização e consolidação de servidores permitem melhor aproveitamento da capacidade computacional, reduzindo desperdícios e aumentando a eficiência energética.

Outro avanço importante é a utilização de IA para otimizar o consumo de energia em data centers. Por meio de algoritmos, é possível ajustar dinamicamente sistemas de refrigeração, distribuição de carga e uso de recursos, garantindo maior eficiência operacional.

O conceito de carbon-aware computing também vem ganhando espaço, propondo a execução de cargas de trabalho em regiões ou horários com menor impacto ambiental, de acordo com a disponibilidade de energia limpa. Essa abordagem integra critérios ambientais às decisões operacionais.

Além disso, cresce a demanda por transparência em relação aos impactos ambientais das operações em nuvem. Provedores têm passado a disponibilizar métricas como pegada de carbono, indicadores de eficiência energética e relatórios alinhados a práticas ESG (ambiental, social e governança), fortalecendo a governança corporativa.

Nesse contexto, a green cloud consolida-se como uma tendência estratégica, ao alinhar inovação tecnológica com responsabilidade ambiental, contribuindo para o desenvolvimento de uma economia digital mais sustentável e consciente.

8.4 Tendências em DevOps

A evolução das práticas de desenvolvimento e operações de software tem ampliado o escopo do DevOps, incorporando novas preocupações que vão além da CI e CD. À medida que os sistemas se tornam mais complexos, distribuídos e fortemente dependentes de ambientes em nuvem, surgem necessidades específicas relacionadas à segurança, à gestão da infraestrutura e ao controle de custos. A partir disso, emergem abordagens complementares que expandem os princípios do DevOps, mantendo sua base cultural e técnica, mas direcionando esforços para áreas estratégicas que exigem maior especialização.

Conceitos como DevSecOps, GitOps e FinOps podem ser compreendidos como desdobramentos naturais dessa evolução. Essas abordagens não substituem o DevOps, mas o fortalecem ao aprofundar dimensões essenciais do ciclo de vida do software: a segurança integrada desde o início, a operação baseada em código e a gestão eficiente de recursos financeiros em ambientes digitais. Ao preservar princípios como automação, colaboração e melhoria contínua, essas práticas contribuem para tornar as organizações mais preparadas para enfrentar os desafios atuais, garantindo mais agilidade na entrega, confiabilidade, governança e sustentabilidade.

Vamos agora estudar cada uma dessas novas abordagens.

8.4.1 DevSecOps

O DevSecOps surge como uma evolução natural do DevOps ao incorporar a segurança como parte integrante de todo o ciclo de desenvolvimento de software. Enquanto abordagens tradicionais tratavam a segurança como uma etapa final, muitas vezes gerando atrasos e retrabalho, o DevSecOps propõe que ela seja considerada desde o início, no planejamento, e aplicada continuamente ao longo de todas as fases do pipeline. Essa mudança reduz riscos e aumenta a confiabilidade das aplicações.

Um dos pilares do DevSecOps é o conceito de shift left, que significa antecipar a segurança para as fases iniciais do desenvolvimento. Isso envolve práticas como análise de código estático, revisão de dependências e definição de políticas de segurança ainda na etapa de codificação. Dessa forma, vulnerabilidades são identificadas mais cedo, quando o custo de correção é significativamente menor.

A automação desempenha um papel central nessa abordagem. Ferramentas de análise de segurança são integradas diretamente aos pipelines de CI e CD, permitindo a execução automática de testes de segurança a cada alteração no código; o que garante que padrões de segurança sejam aplicados de forma consistente, sem depender exclusivamente de revisões manuais.

Outro aspecto importante do DevSecOps é a responsabilidade compartilhada. Em vez de concentrar a segurança em uma equipe isolada, todos os envolvidos (desenvolvedores, operadores e especialistas

em segurança) passam a colaborar ativamente na proteção do sistema. Essa mudança cultural fortalece a conscientização sobre riscos e promove práticas mais seguras no dia a dia.

Além disso, o DevSecOps contribui para a conformidade com normas e regulamentações, como LGPD e outras legislações de proteção de dados. A rastreabilidade proporcionada pelos pipelines e o registro automatizado de atividades facilitam auditorias e demonstram aderência às políticas de segurança.

A adoção dessa abordagem também melhora a resposta a incidentes. Com monitoramento contínuo e integração entre ferramentas, é possível identificar comportamentos suspeitos em tempo real e agir rapidamente para mitigar impactos. Isso é especialmente relevante em ambientes distribuídos e baseados em nuvem. No entanto, implementar DevSecOps não é apenas uma questão de ferramentas, mas de mudança cultural. É necessário promover treinamentos, revisar processos e incentivar a colaboração entre áreas que tradicionalmente atuavam de forma separada. Sem esse alinhamento, a segurança pode continuar sendo tratada como um obstáculo, em vez de um facilitador.

Outro desafio está na escolha e integração das ferramentas adequadas. O excesso de soluções pode gerar complexidade e dificultar a manutenção do pipeline. Por isso, é importante adotar uma estratégia equilibrada, priorizando ferramentas que se integrem bem ao fluxo de trabalho existente. Com o crescimento das ameaças cibernéticas e a crescente dependência de sistemas digitais, o DevSecOps tende a se consolidar como uma prática essencial. Organizações que adotam essa abordagem conseguem entregar software com maior segurança, sem comprometer a agilidade.

Em síntese, o DevSecOps representa a maturidade do DevOps ao incorporar a segurança como um elemento contínuo e estratégico, garantindo que velocidade e proteção caminhem juntas no desenvolvimento de software.

8.4.2 GitOps

Essa abordagem utiliza repositórios Git como a principal fonte de verdade para a gestão de infraestrutura e aplicações. Baseado nos princípios de IaC, ele propõe que todas as configurações e estados desejados do sistema sejam definidos e versionados em repositórios, permitindo maior controle, rastreabilidade e automação.

Nesse modelo, qualquer alteração no sistema é realizada por meio de commits no repositório. Essas mudanças são automaticamente detectadas por ferramentas que sincronizam o estado real do ambiente com o estado desejado definido no Git. Isso elimina a necessidade de intervenções manuais e reduz o risco de inconsistências.

Uma das principais vantagens do GitOps é a rastreabilidade. Como todas as mudanças passam pelo controle de versão, é possível saber exatamente quem fez cada alteração, quando e por quê. Isso facilita auditorias, resolução de problemas e rollback de configurações, tornando o ambiente mais confiável. O GitOps também reforça práticas colaborativas. Alterações são feitas por meio de pull requests, permitindo revisão por pares antes da aplicação. Esse processo melhora a qualidade das configurações e reduz a probabilidade de erros em produção.

A automação é outro elemento central dessa abordagem. Ferramentas específicas monitoram o repositório e aplicam mudanças automaticamente no ambiente, garantindo que o estado desejado seja mantido continuamente. Isso se alinha diretamente aos princípios de CD do DevOps. Além disso, o GitOps é especialmente relevante em ambientes baseados em contêineres e orquestradores como o kubernetes, onde a complexidade da infraestrutura exige mecanismos mais robustos de gerenciamento. Nesse contexto, o GitOps simplifica a operação e aumenta a previsibilidade.

Outro benefício importante é a capacidade de recuperação rápida. Em caso de falhas, basta restaurar uma versão anterior do repositório para retornar o sistema a um estado funcional. Isso reduz significativamente o tempo de indisponibilidade. No entanto, a adoção do GitOps exige disciplina na gestão de repositórios e na definição de padrões. É fundamental garantir que todas as configurações estejam devidamente versionadas e que mudanças fora do fluxo Git sejam evitadas. Também é necessário investir em ferramentas e integração com o ambiente de execução. A escolha inadequada pode comprometer a eficiência da abordagem e gerar dificuldades operacionais.

Conclui-se, portanto, que o GitOps representa uma evolução na forma de gerenciar infraestrutura e aplicações, trazendo maior controle, automação e confiabilidade. Ele reforça os princípios do DevOps ao tratar tudo como código e ao promover um fluxo de trabalho mais transparente e colaborativo.

8.4.3 FinOps

O FinOps surge como uma prática voltada à gestão financeira em ambientes de computação em nuvem, buscando equilibrar custos, desempenho e valor de negócio. Com a crescente adoção de serviços em nuvem, o modelo tradicional de controle de gastos se tornou insuficiente, exigindo abordagens mais dinâmicas e colaborativas.

Diferentemente de modelos tradicionais, em que os custos eram previsíveis e centralizados, a nuvem permite consumo sob demanda, o que pode gerar desperdícios se não houver controle adequado. O FinOps propõe justamente trazer visibilidade e responsabilidade para o uso desses recursos.

Um dos princípios fundamentais do FinOps é a colaboração entre equipes técnicas, financeiras e de negócio. Em vez de decisões isoladas, todos os envolvidos passam a compartilhar a responsabilidade pelos custos, promovendo maior alinhamento com os objetivos organizacionais.

A visibilidade é outro elemento essencial. Ferramentas de monitoramento e análise permitem acompanhar os gastos em tempo real, identificar padrões de consumo e detectar desperdícios. Isso possibilita tomadas de decisão mais rápidas e embasadas.

O FinOps também incentiva a otimização contínua. Isso inclui práticas como desligamento de recursos ociosos, escolha de instâncias mais adequadas e uso de modelos de reserva para reduzir custos. Pequenas melhorias podem gerar economias significativas ao longo do tempo. Além disso, a cultura de accountability é fortalecida. Equipes passam a ser responsáveis pelos custos que geram, o que incentiva o uso mais consciente dos recursos. Essa mudança cultural é fundamental para o sucesso da abordagem.

Outro ponto relevante é a previsibilidade. Apesar da natureza variável da nuvem, o FinOps busca criar mecanismos para estimar custos futuros e evitar surpresas no orçamento. Isso é feito por meio de análises históricas e projeções de uso.

A adoção do FinOps também contribui para a eficiência operacional. Ao otimizar recursos e eliminar desperdícios, as organizações conseguem melhorar o desempenho dos sistemas e reduzir custos simultaneamente.

No entanto, implementar FinOps pode ser desafiador, especialmente em organizações com baixa maturidade em gestão de nuvem. É necessário definir processos, ferramentas e indicadores claros para garantir resultados efetivos. Também é importante evitar uma visão exclusivamente focada em redução de custos, pois o objetivo do FinOps não é apenas economizar, mas maximizar o valor entregue pelo investimento em tecnologia.

Em síntese, o FinOps representa uma evolução na gestão financeira da TI, alinhando custos ao valor de negócio e promovendo uma cultura de responsabilidade compartilhada. Ele complementa o DevOps ao garantir que a eficiência operacional também se reflita na sustentabilidade econômica.



Resumo

A unidade IV abordou de forma abrangente o conceito de DevOps, suas origens, fundamentos, práticas, ferramentas e tendências futuras, além de explorar a evolução do desenvolvimento de software e a integração entre equipes de desenvolvimento e operações. O termo DevOps resulta da combinação das palavras em inglês "development" e "operations", refletindo uma cultura e prática que visa aproximar essas duas áreas, promovendo automação, colaboração contínua e entregas frequentes de software de alta qualidade. Essa abordagem buscou acelerar o ciclo de vida do desenvolvimento, garantindo maior confiabilidade, estabilidade e inovação, especialmente em ambientes de nuvem, onde ciclos de mudança são cada vez mais rápidos e constantes.

A unidade explicou que a evolução do desenvolvimento de software é marcada por diferentes modelos, sendo o cascata o primeiro, caracterizado por uma abordagem linear e sequencial, que, embora estruturada, apresenta limitações significativas frente às demandas de ambientes dinâmicos. Em contrapartida, o modelo ágil surge como uma resposta às restrições do cascata, promovendo uma metodologia iterativa, com entregas frequentes, maior flexibilidade e foco na colaboração com o cliente. O desenvolvimento ágil valoriza a adaptação às mudanças, a comunicação contínua e a entrega de valor incremental, facilitando a inovação e a rápida resposta às necessidades do mercado. Depois, o movimento DevOps amplia essa evolução ao integrar desenvolvimento e operações, criando automação, automonitoramento e entregas contínuas, com foco na cultura de melhoria contínua e na redução do tempo de lançamento de novas funcionalidades.

A abordagem tradicional em cascata, embora adequada para projetos com requisitos bem definidos e ambientes altamente regulamentados, mostra-se rígida e pouco adaptável às mudanças, dificultando a correção de erros precocemente e aumentando os riscos de incompatibilidade com as necessidades do usuário final. Já o método ágil, ao promover ciclos curtos de desenvolvimento, permite maior flexibilidade, maior envolvimento do cliente e uma maior capacidade de adaptação às mudanças de requisitos, o que é fundamental em setores de tecnologia e inovação. O movimento DevOps, por sua vez, reforça essa flexibilidade ao promover uma cultura de colaboração entre equipes, automação de processos e CI, possibilitando entregas mais rápidas, confiáveis e frequentes, além de melhorar a estabilidade dos sistemas em produção.

Fundamentais para a implementação do DevOps são seus princípios, pilares e processos. Os princípios orientam a organização do trabalho em

três principais maneiras: fluxo contínuo de trabalho, feedback rápido e aprendizado constante. O fluxo busca acelerar a passagem de valor desde o desenvolvimento até o cliente, eliminando gargalos e promovendo entregas pequenas e frequentes. O feedback rápido garante que informações sobre o funcionamento do sistema retornem às equipes de forma ágil, possibilitando ajustes e melhorias contínuas. O aprendizado contínuo, por sua vez, incentiva uma cultura de confiança, experimentação e inovação, em que erros são encarados como oportunidades de crescimento, fortalecendo a cultura de melhoria e resiliência.

Os pilares do DevOps incluem a colaboração, que promove o trabalho conjunto entre equipes, e a afinidade, que busca construir vínculos e alinhar metas e responsabilidades. Ferramentas tecnológicas desempenham papel crucial na automação de processos, na gestão de infraestrutura e na monitorização contínua, facilitando a implementação de práticas como CI, CD e automação de testes. O escalonamento é outro aspecto importante, pois permite que as práticas de DevOps evoluam de forma sustentável à medida que a organização cresce ou se adapta às mudanças do mercado, garantindo que os processos, cultura e ferramentas estejam sempre alinhados às necessidades do negócio.

O modelo CALMS, que reúne cultura, automação, práticas enxutas, medição e compartilhamento, serve como uma estrutura para orientar a implementação do DevOps. Cada elemento é fundamental: a cultura promove o ambiente colaborativo; a automação aumenta a eficiência e reduz erros; as práticas enxutas eliminam desperdícios; a medição fornece dados para tomada de decisão; e o compartilhamento fomenta o aprendizado e a inovação coletiva. Essa combinação cria um ciclo de melhoria contínua, em que processos, pessoas e tecnologia trabalham de forma integrada para entregar valor de forma rápida, segura e confiável.

No ciclo de vida do DevOps, etapas como planejamento, codificação, construção, testes, lançamento, implantação, operação e monitoramento são executadas de forma automatizada e contínua. Cada fase é interligada por pipelines de CI e CD, que garantem a consistência e a rastreabilidade das mudanças. Ferramentas específicas, como docker, kubernetes, IaC, monitoramento e logging centralizado, são essenciais para suportar essa automação, permitindo que as equipes respondam rapidamente a incidentes, ajustem recursos e mantenham a estabilidade do sistema.

A integração de ferramentas como Azure DevOps, docker e kubernetes exemplifica como a automação de pipelines possibilita a criação de ambientes de produção confiáveis e escaláveis. A utilização de contêineres e orquestradores facilita a implantação de aplicações em diferentes

ambientes, promovendo maior agilidade e controle. Além disso, a IaC permite que toda a configuração de ambientes seja versionada, auditada e reproduzida de forma automatizada, reduzindo erros humanos e aumentando a confiabilidade do sistema.

Outro aspecto importante na implementação do DevOps é a gestão de configuração, que garante a consistência dos componentes de software, dependências e infraestrutura ao longo de todo o ciclo de vida. Essa prática evita divergências entre ambientes de desenvolvimento, teste e produção, além de facilitar a recuperação de sistemas em caso de falhas.

Ferramentas de IaC, pipelines automatizados e gestão de configuração também suportam práticas emergentes como DevSecOps, GitOps e FinOps. O DevSecOps integra a segurança desde o início do ciclo de desenvolvimento, automatizando testes de vulnerabilidade e revisão de código. O GitOps utiliza repositórios Git como fonte de verdade para gerenciar configurações e infraestrutura, promovendo maior controle e rastreabilidade. Já o FinOps foca na gestão eficiente de custos em ambientes de nuvem, promovendo transparência, responsabilidade e otimização de recursos financeiros.

As tendências futuras apontam para uma maior integração entre computação em nuvem, IA, computação quântica e práticas sustentáveis, como o green cloud. A computação em nuvem continuará evoluindo com modelos como serverless, edge computing e sustentabilidade, promovendo maior eficiência, menor impacto ambiental e maior autonomia na gestão de recursos. A IA será cada vez mais integrada às operações, otimizando processos, detectando ameaças e apoiando a tomada de decisão. A computação quântica, embora ainda em estágio inicial, promete revolucionar áreas que demandam alto poder de processamento, como criptografia, simulação e otimização de processos complexos.

Por fim, a combinação de todas essas tendências reforça a importância de uma cultura de inovação, colaboração e aprendizado contínuo, que é o cerne do DevOps. Organizações que adotam essas práticas e tecnologias estarão mais preparadas para enfrentar os desafios de um mercado cada vez mais digital, competitivo e sustentável, garantindo agilidade, segurança e eficiência na entrega de valor aos seus clientes e stakeholders.



Exercícios

Questão 1. (Funcern 2025, adaptada) DevOps é um conjunto de práticas e de princípios e uma filosofia cultural utilizados amplamente na indústria de software. A respeito do DevOps, avalie as afirmativas.

I – É uma prática aplicada ao desenvolvimento de software na linguagem Java, não se aplicando a projetos que utilizam Python, Go ou outras linguagens.

II – Envolve integração entre equipes de desenvolvimento e operações, garantindo que o software esteja em condições de ser colocado em produção, com pipelines automatizados de build, test e deploy.

III – É uma metodologia estritamente sequencial, similar ao modelo waterfall, em que toda a fase de desenvolvimento deve ser concluída antes que a fase de operações possa começar.

É correto o que se afirma em:

A) II, apenas.

B) III, apenas.

C) I e II, apenas.

D) II e III, apenas.

E) I, II e III.

Resposta correta: alternativa A.

Análise das afirmativas

I – Afirmativa incorreta.

Justificativa: DevOps é um conjunto de práticas independente de linguagem de programação. Desse modo, ele pode ser aplicado com Java, Python, Go, C#, JavaScript etc.

II – Afirmativa correta.

Justificativa: o texto da afirmativa descreve corretamente o DevOps. Esse conjunto de práticas estimula a integração entre as equipes de desenvolvimento e de operações, a automação dos processos, os pipelines de build, testes e deploy, além da CD do produto.

III – Afirmativa incorreta.

Justificativa: o modelo waterfall (cascata) representa um modelo de desenvolvimento de projetos linear e sequencial, em que cada etapa precisa ser concluída antes da próxima começar. Desse modo, o DevOps não segue esse modelo tradicional, pois favorece entregas frequentes, CI, colaboração constante e feedback rápido.

Questão 2. (Cespe-Cebraspe 2025, adaptada) DevSecOps é uma abordagem que integra segurança ao ciclo de vida do desenvolvimento de software, garantindo que vulnerabilidades sejam identificadas e corrigidas desde as primeiras etapas do processo. Uma das características dessa abordagem é a:

- A) Dependência exclusiva de ferramentas externas para a garantia da segurança do software.
- B) Separação total entre desenvolvimento e segurança, que permite que cada equipe trabalhe de forma independente.
- C) Eliminação da necessidade de monitoramento contínuo, pois os testes são realizados apenas antes da implantação.
- D) Restrição do acesso ao código-fonte apenas para a equipe de segurança, o que garante maior proteção.
- E) Automação de testes de segurança, que reduz o tempo necessário para identificação e correção de vulnerabilidades.

Resposta correta: alternativa E.

Análise das alternativas

A) Alternativa incorreta.

Justificativa: o DevSecOps pode utilizar ferramentas externas para suas práticas de segurança, mas não depende exclusivamente delas.

B) Alternativa incorreta.

Justificativa: o DevSecOps faz o contrário: promove a integração entre o desenvolvimento, as operações e a segurança.

C) Alternativa incorreta.

Justificativa: no DevSecOps, o monitoramento contínuo é fundamental. A segurança é tratada durante todo o ciclo de vida da aplicação, inclusive após a implantação.

D) Alternativa incorreta.

Justificativa: a restrição do acesso ao código-fonte apenas para a equipe de segurança não caracteriza o DevSecOps. O modelo favorece a colaboração entre as equipes, o compartilhamento de responsabilidades e a CI, não o isolamento do código.

E) Alternativa correta.

Justificativa: a automação dos testes de segurança é uma das principais características do DevSecOps. Essas ferramentas automatizadas podem executar análises estáticas do código, análises de dependências, testes de vulnerabilidade e varreduras de segurança.

REFERÊNCIAS

- ARRUDA, D. F. *Benefícios e desafios encontrados na adoção de cloud computing*. Caruaru: UPE, 2011. Disponível em: <https://tinyurl.com/5n84kdad>. Acesso em: 26 maio 2026.
- BADO, S. R. de L.; VIONE, C. I. B. Triple botton line: pilares da sustentabilidade empresarial / triple botton line: pillars of corporate sustainability. *Brazilian Journal of Development*, Curitiba, v. 8, n. 3, p. 17507-17513, 2022. Disponível em: <https://tinyurl.com/3evwavda>. Acesso em: 9 jun. 2026.
- BANGAD, S. How to use Amazon Cloudwatch events to monitor application health. *AWS*, 30 ago. 2016. Disponível em: <https://tinyurl.com/4tzuu9wt>. Acesso em: 25 maio 2026.
- BAZZOLI, F. AI inches into the real world and delivers benefits in radiology. *HealthData management*, 12 jan. 2026. Disponível em: <https://tinyurl.com/56hcr2hd>. Acesso em: 29 maio 2026.
- BONOMI, F. et al. Fog computing and its role in the internet of things. In: WORKSHOP ON MOBILE CLOUD COMPUTING, 1., 2012, Helsinki. *Proceedings [...]*. New York: ACM, 2012. p. 13-16. Disponível em: <https://tinyurl.com/zym5emp3>. Acesso em: 20 maio 2026.
- BRUNO, A.; JORDAN, S. *CCDA 640-864: official cert guide*. Indianapolis: Cisco Press, 2011. Disponível em: <https://tinyurl.com/252rh5mj>. Acesso em: 27 maio 2026.
- THE CLOUD value chain: maximizing the benefits of a strategic digital transformation blueprint. *GlobalData*, 14 set. 2022. Disponível em: <https://tinyurl.com/yc4bu2t>. Acesso em: 1 jun. 2026.
- COELHO NETO, N. J. Segurança em redes de computadores – triade C.I.A. *Lindekin*, 13 jun. 2023. Disponível em: <https://tinyurl.com/4h2zh3ba>. Acesso em: 1 jun. 2026.
- DINO, D. Pesquisa revela que 77% dos gamers brasileiros preferem a nuvem aos consoles. *ABRACD*, 5 ago. 2022. Disponível em: <https://tinyurl.com/ru94bapb>. Acesso em: 29 maio 2026.
- FERREIRA, O. M. D. *O nível de implementação do cloud computing nas empresas*. Portugal: Universidade Portucalense, 2012.
- GOTTEMS, L. Computação em nuvem transforma o agronegócio. *AGROLINK*, 17 dez. 2024. Disponível em: <https://tinyurl.com/e68xzdbv>. Acesso em: 29 maio 2026.
- KOLBE JÚNIOR, A. *Computação em nuvem*. São Paulo: Contentus, 2020. E-book. Disponível em: <https://tinyurl.com/seywnx6t>. Acesso em: 18 maio 2026.
- LANDGRAF, L. Drones ganham espaço nas lavouras brasileiras e viram opção para agricultores e prestadores de serviço. *Embrapa*, 15 jul. 2025. Disponível em: <https://tinyurl.com/2vw5p5r2>. Acesso em: 29 maio 2026.

LOPES, T.; SOUZA NETO, J. Nuvem pública na APF: recomendações na contratação. In: SIMPÓSIO DE EXCELÊNCIA EM GESTÃO E TECNOLOGIA, 12., 2015, Resende. *Anais [...]*. Resende: Seget, 2015.

MALTA, D. P. L. N. *et al.* O papel da computação em nuvem na gestão e acessibilidade educacional. *Missioneira*, v. 27, n. 9, p. 409-423, 2025. Disponível em: <https://tinyurl.com/ebt9edwm>. Acesso em: 29 maio 2026.

MANIFESTO para desenvolvimento ágil de software. *Agile Manifesto*, [s.d.]. Disponível em: <https://tinyurl.com/3z3xj7dy>. Acesso em: 1 jun. 2026.

MELL, P.; GRANCE, T. The NIST definition of cloud computing: recommendations of the National Institute of Standards and Technology. NIST, 2011. Disponível em: <https://tinyurl.com/2x883fx7>. Acesso em: 1 jun. 2026.

MIGNON, A. S. *DevOps, continuous integration, quality and delivery*. São Paulo: Senac, 2025.

MUNIZ, A. *et al.* *Jornada Azure DevOps: unindo teoria e prática com o objetivo de acelerar o aprendizado do Azure DevOps para quem está iniciando*. Rio de Janeiro: Brasport, 2021.

NIETSCHE, M. Integração ERP usando API: parte 1. *interGATE*, 10 jul. 2019. Disponível em: <https://tinyurl.com/yrnby65j>. Acesso em: 29 maio 2026.

PINTO, O. P. *Curso de cloud computing: dominando AWS, Azure e Google Cloud: aprenda na prática os três principais provedores de nuvem*. Orlando, 2025.

O QUE é segurança em nuvem? Benefícios e melhores práticas. *Fortinet*, [s.d.]. Disponível em: <https://tinyurl.com/nvzhwj9p>. Acesso em: 27 maio 2026.

RAPÔSO, C. F. L.; COSTA JUNIOR, O. G.; FERREIRA, A. S. Impactos da computação em nuvem na arquitetura de software: uma análise de literatura. *Revista Tópicos*, v. 2, n. 14, p. 1-19, 2024. Disponível em: <https://tinyurl.com/37a58pae>. Acesso em: 20 maio 2026.

ROCHA, W. *Computação em nuvem e seus benefícios da elasticidade*. 2020. Trabalho de Conclusão de Curso (Bacharel em Sistema da Informação) – Universidade Anhanguera, São Paulo, 2020. Disponível em: <https://tinyurl.com/3az4mku2>. Acesso em: 1 jun. 2026.

SACOMANO J. B. *et al.* *Indústria 4.0*. São Paulo: Blucher, 2018.

SANCHEZ, L. *et al.* SmartSantander: IoT experimentation over a smart city testbed. *Computer Networks*, [s.l.], jan. 2013. Disponível em: <https://tinyurl.com/y4x88tra>. Acesso em: 6 jun. 2026.

SANTOS, J. C. D. *Computação em nuvem e cloud platform: design de soluções em nuvem*. Recife: Grupo Ser, 2022.



A series of horizontal lines for writing, consisting of 30 evenly spaced lines across the page.



Informações:
www.sepi.unip.br ou 0800 010 9000